ShuffleSnap: strided windowed exact reassignment for point-to-grid layout, evaluated against certified optima

Kyle McDonald

October 6, 2026

Abstract

Snapping each point of a 2D embedding (such as a UMAP layout) to its own cell of a grid, while minimizing total squared displacement, is a linear assignment problem. Dense solvers cannot handle it beyond a few tens of thousands of points. We develop and test ShuffleSnap, a local search inspired by Shellsort. At a decreasing sequence of strides, it splits the grid into interleaved subgrids and solves small exact assignment problems in shifted 8×8 windows of each. Every window solve starts from dual potentials derived in closed form from per-axis optimal transport, which makes it 2–5× faster than a standard Hungarian solver with the same results.

To measure true optimality gaps, we built a certified exact solver. It solves min-cost flow on sparse candidate edges and proves the result optimal for the full, dense problem with a dual bound evaluated over all cells. Started from ShuffleSnap, it proved optimality on every instance we tried, up to 10^6 points.

On held-out instances with 10^5 points, ShuffleSnap is within 0.06% of the optimum on nine of ten distributions, in a median of 2.3 s on one core or 0.19 s on 24 threads. Its fast and default presets take about as long as BSP-OT with 16 and 64 plans, and at equal time they are closer to the optimum on eight and nine of the ten distributions. BSP-OT with 64 plans is 3.7–36% above the optimum on five of them, and RasterFairy is 17–425% above it on seven. SciPy's dense exact solver already needs up to 6 minutes at 10^4 points. On one workstation, ShuffleSnap also handled a billion points: using 53 GiB, it passed a six-hour run of an earlier implementation after 44 minutes and converged after 1 h 43 min, while a single-plan BSP-OT ended 8.2% above it.

The method's limits are structural. Its error consists of long closed loops of one-cell shifts that no window can remove, and we construct inputs on which its relative gap is unbounded. Uniform points remain the hardest common input (0.8% at 10^5), yet random starts give a predictable result: on 50 clouds of 10^4 uniform points with 50 random starts each, every cloud showed nearly the same right-skewed, nearly log-normal spread of absolute excess, and 90% of the 2,500 runs were 0.55–1.38% above the optimum (mean 0.91%). Bisection starts make results worse, and polishing at coarser strides changes almost nothing. Warm-started from ShuffleSnap, the certified solver reaches proven optima in seconds to minutes at 10^5 points.

On two real UMAP embeddings, ShuffleSnap is within 0.002% of the optimal displacement. There, displacement and neighbourhood preservation diverge: BSP-OT is 2–7% above the optimal displacement but halves the fraction of preserved grid neighbours. We release the implementation, all experiment code and the raw results.

1 Introduction

A common way to show a large image or document collection is to embed the items in the plane (for example with t-SNE [44] or UMAP [29]) and then snap every item to its own cell of a regular

grid, so that thumbnails tile the screen without overlap. Snapping is an assignment problem: each point must receive a distinct cell, and we would like points to move as little as possible. Measured as total squared displacement, the best snapping is a linear assignment problem (LAP), and also a discrete optimal-transport problem with quadratic cost [34].

For a few thousand points, practitioners solve this LAP exactly with a dense Jonker–Volgenant or Hungarian solver [19, 26–28]. The dense cost matrix needs $8N^2$ bytes, which is 80 GB at $N = 100{,}000$, so these solvers stop being usable somewhere between 10,000 and 30,000 points. Beyond that, people turn to recursive-splitting heuristics such as RasterFairy [21] or to fast approximate transport such as BSP-OT [12].

This paper develops and evaluates ShuffleSnap, a simple heuristic for this regime, taking its name from Shellsort [41]. Shellsort sorts interleaved subsequences at a decreasing sequence of gaps. ShuffleSnap does the same in 2D: at stride s it splits the grid into s^2 interleaved subgrids and solves small *exact* assignment problems inside 8×8 windows of each subgrid, dividing s by $\sqrt{2}$ until it reaches 1 (Figure 1). All work is at full resolution; nothing is aggregated or downsampled.

Every gap we report is measured against a proven optimum on the same instance, and we report where the method fails as well as where it works. The main contributions are:

- **A careful implementation** with documented rules for boundaries, rectangular grids, surplus cells and masks. It is a parallel C++ kernel with a Python API whose results do not depend on the number of threads, and three presets: *fast* (5×5 windows), *balanced* (8×8, the default) and *quality* (12×12).
- **A geometric window solver.** Each window is solved exactly, starting from dual potentials derived in closed form from per-axis optimal transport between the window's points and cells. This removes the conflicts that make clustered windows slow for standard solvers. It is 5.0–5.4× faster than the Hungarian method with a greedy start on clustered data and 2–2.4× faster on uniform points, with the same results. The saved time buys larger windows at the same running time.
- **A certified exact solver that reaches 10^6 points.** It solves min-cost flow on sparse candidate edges, then prices every edge of the *dense* problem through a Lagrangian bound, adding edges until the bound meets the primal cost. It gives true optimality gaps, not progress relative to our own best answer, for all 232 benchmark instances (up to 200,000 points) and for uniform instances up to 10^6 points. Where SciPy's dense solver fits, both found the same optimum on every instance: 600 small ones and all ten distributions at 10^4 points. It turned out to be a strong practical baseline as well.
- **Measured design choices.** The presets were chosen on a development set and every claim is evaluated on held-out instances. Window size is the main quality knob, and dividing the stride by $\sqrt{2}$ is best or close to best at every window size. Several intuitive ideas did not help:
  - bisection starts make results *worse*, and so do row-sorted starts on most inputs;
  - polishing at coarser strides changes almost nothing, and merging independent runs helps less than a larger window;
  - a Jonker–Volgenant window solver is slower, and an auction warm start helps less than the geometric one.
- **A diagnosis of the remaining error.** The gap to the optimum is concentrated in a few long closed loops of points, each displaced by about one cell. No window smaller than a loop can remove it. Masks with concave bays and grids with surplus cells expose the same

mechanism at a larger scale. From random starts the result is predictable: over 50 uniform point sets and 50 random starts each, the excess per point follows nearly the same, nearly log-normal distribution on every set, and the start, not the set, causes 96% of its variance.

- **A benchmark against exact LAP, RasterFairy and BSP-OT.** We first verified each comparator's capabilities: we found and worked around coordinate truncation in the PyPI release of RasterFairy, and used its much stronger upstream version. The fast and default presets take about as long as BSP-OT with 16 and 64 plans, so the two methods can be compared at equal time: ShuffleSnap is closer to the optimum on eight and nine of ten distributions. The benchmark covers varied and adversarial distributions, masks, prime point counts, two real UMAP embeddings with 70,000 points each, multiple seeds, matched-budget curves, and scaling to 10^6 points, plus one run on 10^9 points. We report cost, time, peak memory, and neighbourhood preservation, and we keep displacement and neighbourhood measures separate.

The implementation is on PyPI as `shufflesnap` (`github.com/kylemcdonald/shufflesnap`); this paper's experiment code, raw results and figure scripts are at `github.com/kylemcdonald/shufflesnap-paper`.

2 Problem

Cells. A target is a boolean mask on a $W \times H$ lattice. Lattice position (x, y) is column x and row y, and usable positions are the cells $c_1, \ldots, c_M$. Cell centers default to the integer coordinates, so one unit is one cell pitch. Anisotropic pitches (e.g. for 4:3 thumbnails) scale the centers but leave the lattice unchanged. A full rectangle, a rectangle whose last row is only partly usable, a disk, a ring, and the shape of a letter are all masks.

Points and normalization. Input points $q_1, \ldots, q_N$ $(N \le M)$ are mapped into the cell frame before anything else. Unless stated otherwise, each axis of the point bounding box is mapped affinely onto the bounding box of the usable cell centers (*bbox* normalization); p_i denotes the mapped points. The mapping is part of the problem: in every comparison, all methods receive identical p_i and identical cell centers.

Objective. We seek an injective assignment $\sigma : \{1..N\} \to \{1..M\}$ minimizing

$$C(\sigma) = \sum_{i=1}^{N} \|p_i - c_{\sigma(i)}\|^2. \tag{1}$$

When $N = M$ this is a bijection. When $N < M$ the solver must also decide which $M - N$ cells stay empty. Unlike the bijective case, the cell term $\sum_i \|c_{\sigma(i)}\|^2$ then depends on the choice, so the surplus case is not just a relabelling of the bijective one.

A certificate of optimality. Equation (1) is the linear program $\min \sum_{ij} c_{ij} x_{ij}$ with $\sum_j x_{ij} = 1$ for every point, $\sum_i x_{ij} \le 1$ for every cell, and $x \ge 0$, where $c_{ij} = \|p_i - c_j\|^2$. Its constraint matrix is totally unimodular. We use the following elementary bound.

Proposition 1. *For any cell potentials $v \in \mathbb{R}^M$,*

$$\mathrm{LB}(v) = \sum_{i=1}^{N} \min_j \big(c_{ij} - v_j\big) + \sum_{j=1}^{M} v_j - (M - N) \max_j v_j$$

is a lower bound on the optimal cost. The bound is tight (equals the optimum) for optimal dual potentials.

Proof. Let $\bar{v} = \max_j v_j$ and $v'_j = v_j - \bar{v} \le 0$. Let $u_i = \min_j(c_{ij} - v'_j)$. Then (u, v') is feasible for the dual $\max \sum_i u_i + \sum_j v'_j$ subject to $u_i + v'_j \le c_{ij}$ and $v' \le 0$, and its objective equals $\mathrm{LB}(v)$ (shifting all potentials by $\bar{v}$ changes every u_i by $+\bar{v}$ and the cell sum by $-M\bar{v}$). Weak duality gives the bound. Strong duality of the LP gives tightness. □

The bound needs a minimum over *all* M cells for every point. It does not depend on how v was obtained, so $C(\sigma) - \mathrm{LB}(v)$ certifies the gap of any assignment σ. We use it for every "excess over optimum" number in this paper, and we never measure progress relative to our own final cost.

Reporting error. We report the *relative excess* $C(\sigma)/C^\star - 1$ and the *absolute excess per point* $(C(\sigma) - C^\star)/N$ in cell units squared, where $C^\star$ is a certified optimum. Both are needed. Inputs that already lie almost on the cells have a tiny $C^\star$, which makes relative numbers explode. Inputs dominated by outliers (heavy tails) have a huge $C^\star$, which makes every method look perfect in relative terms.

3 Related work

Exact assignment. The Hungarian method [23, 32] and shortest-augmenting-path variants such as Jonker–Volgenant (JV) [18] solve dense $N \times N$ problems in $O(N^3)$ time and $O(N^2)$ memory. SciPy's `linear_sum_assignment` implements a modified JV without initialization [8, 47]. The `lap` and `lapjv` packages provide dense JV with heuristic initialization (`lap` also provides a sparse variant) [19, 27]. Auction algorithms [3, 4] and cost-scaling push–relabel [15] handle sparse problems, and Volgenant's core-oriented method solves a dense problem on a sparse core of its cost matrix [48]. The min-cost-flow solver in OR-Tools, which we use, is a cost-scaling push–relabel method [16, 33]. Burkard et al. [7] survey the field. Geometric bipartite matching admits near-linear ε-approximations in theory [38, 40]. We know of no practical implementations aimed at this use case.

Optimal transport. With quadratic cost, optimal maps are gradients of convex functions [6]. Least-squares assignments with prescribed cluster sizes are exactly those induced by power diagrams [1], the geometric fact behind semi-discrete solvers [25, 31] and behind the potentials in our certificate. Multiscale methods solve coarsened problems and refine them [14, 31, 39]. Schmitzer's sparse multiscale algorithm in particular alternates sparse subproblems with global optimality checks [39], as our exact solver does. Multiscale solvers also warm-start each level with duals from the coarser one [31, 39]; our window solver instead derives starting duals for every small subproblem in closed form, from the per-axis sorted matching that sliced transport also builds on [5, 37]. Entropic regularization [9] and sliced transport [5, 37] trade exactness for speed; see Peyré and Cuturi [34] for a survey. BSP-OT [12] builds many randomized binary space partition matchings in $O(N \log N)$ and merges them with a cycle-based rule. Its official code [13] has Python bindings [10, 43]. The BSP-OT paper does not evaluate uniform grid targets.

Grid layouts for visualization. RasterFairy [21] recursively slices the point cloud and the target raster in matching proportions, and supports masks and hexagonal rasters. Its current upstream version also solves small leaf blocks exactly (Section 6). IsoMatch [11] projects with Isomap and then solves a dense Hungarian problem, which it names as its bottleneck. Kernelized sorting [35, 36], self-organizing maps [22], the Self-Sorting Map [42], and Linear Assignment Sorting and its fast variant FLAS [2] arrange high-dimensional items directly on a grid. LAS repeatedly solves full JV problems between features and smoothed map vectors. Gridfit [20] and DGrid [17] remove overlap in scatterplots by hierarchical partitioning. These methods mostly optimize neighbourhood preservation, and they measure it with trustworthiness and continuity [45, 46] or the Distance Preservation Quality [2]. We keep that goal distinct from displacement: the LAP objective only minimizes how far points move.

Sorting networks and Shellsort. Shellsort [41] sorts with insertion-sort passes over a decreasing sequence of gaps, so early passes move elements far and late passes move them little. ShuffleSnap borrows this schedule. Unlike Shellsort's final gap-1 pass, however, its final stride-1 stage is not guaranteed to reach the optimum. That difference turns out to be the heart of the matter (Section 7.8).

4 ShuffleSnap

4.1 The algorithm

ShuffleSnap has two main parameters, the window side w and the stride ratio r. The default is $w = 8$ and $r = \sqrt{2}$ (Section 4.4).

1. **Start.** Place the N points on a uniformly random subset of N cells, as a seeded random permutation.
2. **Strided windows.** At stride (s_x, s_y), lattice positions with the same residues ($x \bmod s_x$, $y \bmod s_y$) form a subgrid, for $s_x s_y$ subgrids in total. In its own coordinates each subgrid is tiled with $w \times w$ windows. A window therefore contains at most w^2 cells (64 by default), spaced s apart on the lattice and spanning $(w-1)s + 1$ cells per axis.
3. **Exact reassignment.** In every window, the points currently occupying its cells are reassigned among the same cells, empty ones included, by solving the dense linear assignment on squared distances to the original (normalized) coordinates. The window's cell set never changes, so the global assignment stays injective.
4. **Shifted tilings.** The tiling is repeated with shifts $(0, 0)$, $(0, w/2)$, $(w/2, 0)$ and $(w/2, w/2)$ in subgrid coordinates. These four phases form one round. Windows in one phase are disjoint and are solved in parallel.
5. **Schedule.** On an axis of length L, the first stride is the smallest power of two with $ws \geq L$, so that the first windows span the grid. After one round, both strides are divided by r and rounded, with repeats removed and never below 1. Stride-1 rounds then repeat until a full round moves no point, or until a time budget runs out.

With $r = 2$ the strides halve, and every level quarters the area a window spans. With $r = \sqrt{2}$ the area halves from one level to the next, which roughly doubles the number of levels. We chose $\sqrt{2}$ experimentally (Section 7.7).

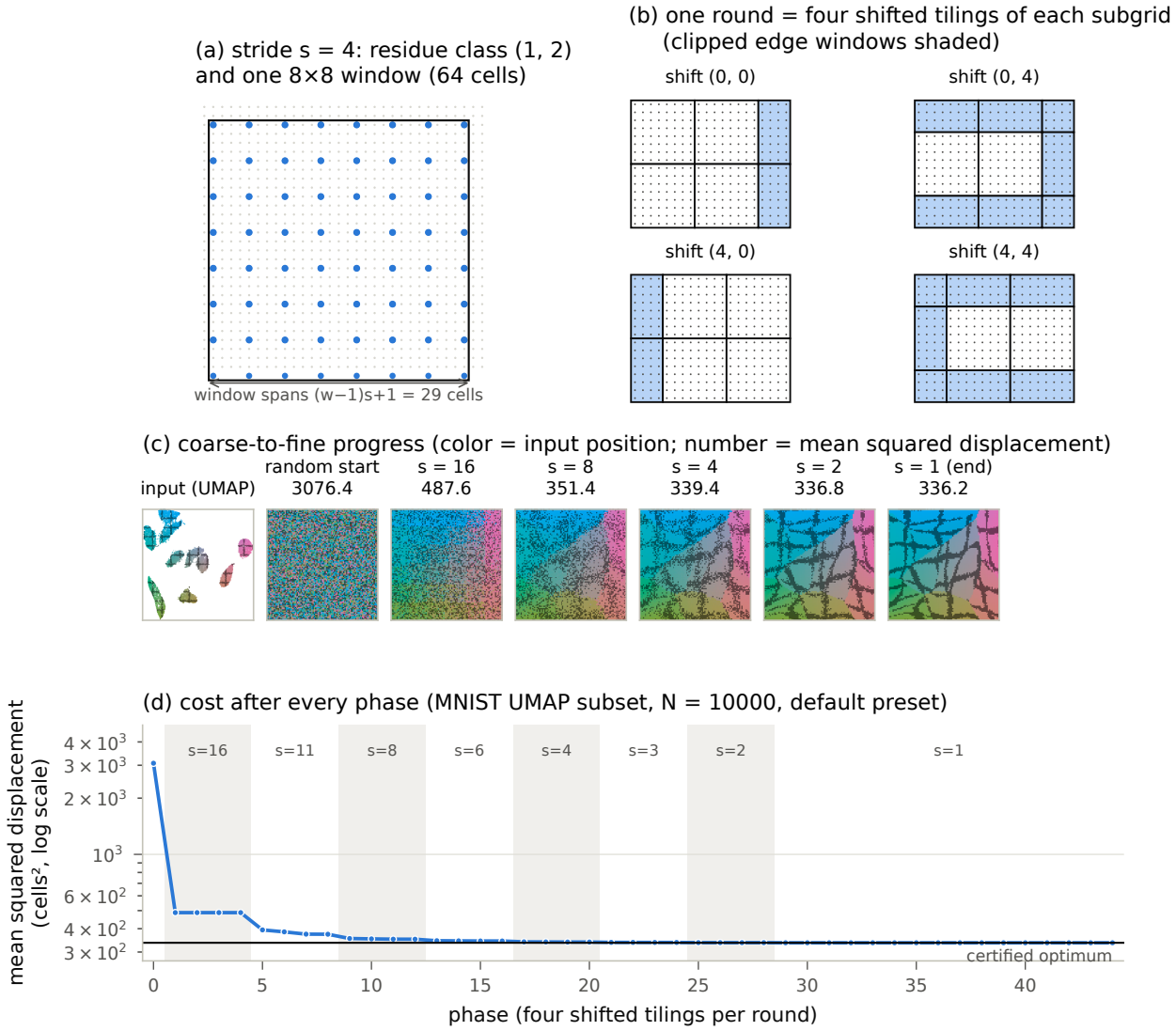


Figure 1: How ShuffleSnap works. (a) At stride $s = 4$ the lattice splits into $s^2 = 16$ interleaved residue subgrids. Blue marks one of them. An 8×8 window of that subgrid (filled blue) holds 64 cells spaced 4 apart and spans 29 cells. The occupants of those 64 cells are reassigned among the same cells by an exact linear assignment. (b) Each round tiles every subgrid four times, with the tiling shifted by half a window in x, y, or both, so that window borders move. Windows that cross the subgrid edge are clipped (shaded), never dropped, on all four sides. (c) Progress on a 10,000-point subset of a UMAP embedding of MNIST. Each point is colored by its *input* position, and the dark bands are the lines of a square grid drawn on the input (graph paper). Both travel with the points, so a smooth color field and unbroken lines mean the input neighbourhood structure survived; every second stride is shown. (d) Mean squared displacement after every phase. The coarse strides remove most of the cost; the stride-1 rounds that follow change little.

Window solver and acceptance. Each window is a rectangular problem: k occupants and $m \geq k$ cells, with $m \leq w^2$. We solve it exactly by shortest augmenting paths with dual potentials (the Hungarian method, $O(k^2 m)$ time), started from duals built from the window's geometry (Section 4.2). A new window assignment replaces the old one only if it is cheaper by more than a relative 10^{-12}. Because the old assignment is feasible for the window problem, every accepted change strictly lowers the global cost by at least that margin. Hence the cost sequence is monotone, the search terminates, and ties (e.g. duplicated points) never cause cycling.

4.2 A geometric window solver

Almost all of the running time is spent in window solves, so the choice of solver matters. Generic LAP solvers are slow on these windows for a geometric reason. On clustered data, and at coarse strides in particular, a window's occupants often come from a dense cluster that lies far from most of the window's cells. Seen from far away, every point prefers the same corner cell. The usual greedy start (match each row to its cheapest free column) then matches little, and the shortest augmenting paths become long. We captured about 79,000 windows from runs with 8×8 windows and $r = \sqrt{2}$ (20,000 points; unchanged windows skipped as in the kernel). In 96% of the MNIST windows with at least 16 points, the points' cheapest cells numbered at most half as many as the points (median: one cheapest cell per 3.6 points). On uniform points this happened in 1% of the windows.

The squared-distance structure offers a better start. For the cost $|p - q|^2$, optimal duals on the cells are a discretized Brenier potential [6]: $v(q) = |q|^2 - 2\psi(q)$ with ψ convex, and $\nabla\psi$ maps cells to the points that occupy them. We use the potential of a map that is optimal per axis. On one axis, the optimal transport between the points' and the cells' coordinates is the sorted order, and its potential is piecewise linear: between consecutive distinct cell coordinates, ψ increases at a rate equal to the point quantile at that cumulative cell mass. Summing the two axes gives separable starting duals in $O(k \log k + m \log m)$ time. They move a distant cluster onto the window and spread it over the window's rows and columns, which removes most of the conflicts. The solver then proceeds in four exact steps:

1. row reduction under the starting duals, and a greedy match of every row whose cheapest column is still free;
2. a column reduction, which keeps the duals feasible and makes every cell tight for at least one point;
3. Kuhn's augmenting paths restricted to tight (zero reduced cost) edges, for the rows still unmatched;
4. shortest augmenting paths for the rest (Dijkstra over a shrinking list of unscanned columns with lazy dual updates, ties broken towards free columns, as in SciPy's solver [8]).

Arbitrary starting duals are valid only for a square problem. A window with empty cells is therefore either padded with zero-cost rows and solved from the geometric start, or, if fewer than half of its rows compete for a cell under zero duals (as on near-uniform data), solved directly as a rectangular problem from zero duals. Padding such a window would cost one augmenting path per empty cell.

On the captured windows, the solver returned the optimum on every window. It was 5.1–5.6× faster than the Hungarian method with a greedy start on MNIST and mixture windows, and 1.9× faster on uniform ones. In full runs with the default 8×8 windows it is 5.1–5.3× faster on clustered inputs and 2.0–2.1× faster on uniform points, on 1 or 24 threads (Section 7.7). Its results are identical to the other exact solvers' up to ties.

4.3 Implementation

The kernel is C++17, exposed to Python with nanobind, and parallelized with OpenMP over the windows of a phase. Each thread keeps its own scratch buffers. Because windows in a phase are disjoint and each solve is deterministic, the result is bit-identical for any thread count; a test checks this. We also track, for every cell, the last phase that changed its occupant. A window whose cells are unchanged since it was last solved in the same stage is already optimal, and skipping it does not change the result (also tested). This matters in the stride-1 convergence rounds, where most windows are untouched. The cost is recomputed exactly after every phase, which gives an anytime trace, and a time budget is checked between windows. A serial reference of about 55 lines of NumPy and SciPy (Appendix B) returns exactly the kernel's assignments and serves as an executable specification. There is no just-in-time compilation, so nothing is compiled at run time. Compiling the extension takes 2.6 s with parallel Ninja (8.1 s on one core). A complete isolated wheel build, including creating the build environment as `pip install` does from source, takes about 35 s.

Work per level. A phase at stride s has about M/w^2 windows (plus clipped ones), each an $O(w^6)$ Hungarian solve in the worst case. The work per phase is therefore $O(Mw^4)$, independent of s, and a schedule with ℓ levels costs $O(\ell M w^4)$ plus the convergence rounds. With $\ell = O(\log L)$ this is $O(M \log M)$ for fixed w. The constant depends strongly on the data: windows far from optimal need long augmenting paths.

4.4 Choosing the parameters

We chose every setting on a development set (instance seed 0: 23 instances of 10,000 to 100,000 points, including masks and surplus cells, five start seeds each) and report every claim on held-out seeds (Section 7.7).

- **Window size** is the main quality knob. We tried window sides from 6 to 24 with four stride ratios (35 configurations). Every step up in window size lowered the excess, and the gains did not level off: from 8×8 to 12×12 to 16×16 windows, each step cost 2–2.6 times the time and removed one half to two thirds of the remaining excess. There is no sweet spot, so we place three presets along the curve, two of them where they can be compared with BSP-OT at equal time: *fast* (5×5 windows) takes about as long as BSP-OT with 16 plans, its default, and *balanced* (8×8, our default) about as long as BSP-OT with 64 plans, on one core at 10^5 points. *Quality* (12×12) is one step further. For the fast preset, 5×5 windows with ratio $\sqrt{2}$ beat 6×6 windows with halving strides, which take the same time, on five of seven development instances. Windows up to 32×32 can be set explicitly. The cost of a window solve grows as w^6 in the worst case, but a phase has M/w^2 windows and most of them are nearly sorted or skipped; in our runs the total time grew as about $w^{1.9}$ (uniform) to $w^{2.8}$ (clustered data). Using small windows at coarse strides and large ones at stride 1 was worse than using large windows throughout.
- **Stride ratio.** Dividing strides by $\sqrt{2}$ instead of 2 (rounded, duplicates removed) roughly doubles the number of levels, each at about the same cost, and was on or next to the best time–quality trade-off at every window size. Ratios of 1.68 and 1.19 were close; 2 and 3 were clearly worse.
- **Start.** Structured starts are mostly *worse* than a random permutation: bisection (the recursive median split used by RasterFairy-like methods) on every input, and row sorting on

most. They create straight seams that the local windows cannot repair. The random start is a decorrelating device, not a lazy default.

- **Solver.** All solvers we tried are exact, so they change only time. JV with augmenting row reduction is slower than the Hungarian method on these windows, because squared distances create many near-ties. An ε-scaled auction whose prices warm-start the same exact shortest-path finish is 2–3× faster on clustered data but not on uniform points. The geometric solver (Section 4.2) is the fastest on every input. Variants that did not pay off, measured on the captured windows: starting duals from an affine or a full-covariance Gaussian map (both worse than the per-axis quantile map), duals integrated from the current assignment (better on clustered windows, worse on uniform ones, and choosing per window cost more than it saved), an auction refinement of the geometric start, repeated row and column reductions, and a branch-free scan of all columns in the shortest-path step.
- **Post-processing.** Extra stride-2 and stride-1 rounds after convergence changed nothing on seven of nine held-out instances (a stride-2 window cannot move a point by one cell). Merging independent runs component by component, the rule BSP-OT uses, helps less per unit time than a larger window.

5 A certified exact solver for large instances

Excess over the optimum is only meaningful with a trustworthy optimum. At $N = 100{,}000$ a dense matrix does not fit in memory. SciPy's sparse LAPJV on a 16-nearest-cell graph did not finish a 10,000-point instance in eight minutes; we later found that such graphs are usually *infeasible*, because some cells are not among any point's 16 nearest. We therefore built a solver that never forms the dense problem, yet certifies optimality against it.

1. **Candidates.** Give each point its 12 nearest cells, plus the 5×5 lattice neighbourhood of its cell in a starting assignment. Including the starting assignment guarantees that a perfect matching exists.
2. **Sparse solve.** Solve the min-cost flow (points → cells → sink, unit capacities) with OR-Tools' cost-scaling solver. The solver needs integer costs, so we scale by $S = 2^{56}/((N{+}M{+}2)\max c)$ and round. (We found by probing that a scale of $2^{58}/((N{+}M)\max c)$ is accepted and $2^{60}/((N{+}M)\max c)$ is rejected with `BAD_COST_RANGE`.)
3. **Potentials.** For the sparse optimum σ, build a graph on cells with an arc $\sigma(i) \to j$ of weight $c_{ij} - c_{i\sigma(i)}$ for every candidate edge. Shortest-path distances from a virtual source joined to all cells give potentials v that satisfy complementary slackness on every candidate edge, with $v = 0$ on empty cells. We compute them in exact integer arithmetic with a label-correcting algorithm.
4. **Global pricing.** Evaluate $\mathrm{LB}(v)$ (Proposition 1) over *all* cells. Cells are grouped into 16×16 blocks, and a block is scanned only if $\mathrm{dist}^2(p_i, \mathrm{block}) - \max_{\mathrm{block}} v$ could beat the best value found so far. The pass also lists every cell whose reduced cost undercuts the point's current cell. If the bound meets the cost, stop. Otherwise add those edges, plus the 3×3 neighbourhood of each such point's best-priced cell, and go back to step 2.

The certificate is the gap $C(\sigma) - \mathrm{LB}(v)$, computed in floating point from real-valued costs. It is valid whatever errors occur in steps 1–3. Integer rounding can only loosen it, by at most $4N/S$ in total; we count an instance as certified when the gap is at most $\max(10^{-9}C, 4N/S)$. The result is

therefore the exact optimum of the dense problem, whatever candidate edges it started from; the start affects only the time. Wherever SciPy's dense solver fits, both found the same optimum: on 600 small instances ($N \leq 2{,}500$, five distributions, three grid types) to relative 10^{-9} (Section 7.1), and on all ten distributions at $N = 10^4$ to relative 10^{-10}.

A pitfall. Our first version flagged a "negative cycle" on some clustered instances with masks. It declared a cycle once a node had been queued more than n times, which is only valid for FIFO Bellman–Ford, not with the small-label-first queue we use. The fix is to verify a suspected cycle by walking parent pointers. A regression test covers it.

Cost. Started from ShuffleSnap, the solver reached a certificate on every instance we gave it: the 232 reference instances (up to 200,000 points), every timed run, and uniform instances up to 10^6 points. Started from bisection, it failed on 4 of 10 instances at 10^5 points (Section 7.2). The number of pricing rounds was 1 for uniform inputs, up to about 8 for clustered, curved or masked inputs, and up to 13 for the input with 16 heavily duplicated sites. Massive ties are the slowest case: one such 100,000-point instance took 34 minutes, while the median 100,000-point reference took under 3 minutes (six references were computed in parallel). Section 7 reports its running time and memory next to the heuristics. The solver needs a feasible starting assignment. We use ShuffleSnap's default preset, and at 10^5 points we also run it from a bisection start, to separate the two methods' contributions.

6 Experimental setup

Hardware and software. All runs used one workstation: an Intel Core i9-12900KS (8 performance and 8 efficiency cores, 24 hardware threads), 62 GB RAM, Ubuntu 24.04 (Linux 6.8), Python 3.12.3, NumPy 2.5.3, SciPy 1.18.1, OR-Tools 9.15, GCC 13.3 with `-O3` and no `-march=native` (portable build). The GPU was not used. Exact versions and commit hashes are stored with every raw result. *Single-thread* runs were pinned to one performance core (`taskset -c 8`) with all thread pools set to 1. *Multi-thread* runs used all 24 hardware threads without pinning. For ShuffleSnap this was faster on this machine than using only the 16 performance-core threads (Section 7.10). The machine also ran unrelated background services (load average about 1). Benchmarks ran sequentially, only after all reference optima had been computed.

Timing and memory boundaries. Every run is a fresh process. The timer brackets exactly one call of the method adapter. It includes all method-specific work: cost matrices, trees, starts, the method's own normalization, and format conversion. It excludes instance generation, the common normalization, validation, and metrics. Before the timed call, each method is run once on a 256-point instance. This takes imports, lazy initialization and thread-pool start-up out of the timing; that warm-up time is recorded separately and is below 0.3 s for every method. None of the methods uses just-in-time compilation. ShuffleSnap's one-time C++ build is not part of any run: 2.6 s to compile, or about 35 s for an isolated wheel build. *Peak memory* is the rise of the process's peak resident set size during the call: the kernel counter is reset through `/proc/self/clear_refs` just before the call, and the resident size at that moment is subtracted. It therefore measures what the method needs on top of its inputs.

Seeds and held-out evaluation. All design decisions were made on instance seed 0. Reported results use instance seeds 1–3, which were not used for tuning; the study of spread across inputs

(Section 7.9) uses fresh seeds 100–149. For the real embeddings, which have one fixed instance each, the seeds vary only the methods' own randomness.

Data. Synthetic distributions: uniform square, isotropic Gaussian, a 10-component anisotropic Gaussian mixture (*gmm*, cluster structure like typical embeddings), two clusters with 90/10 mass (*unequal2*), a thin ring, a noisy spiral (a 1D manifold), a checkerboard density with a 4:1 contrast, Student-t with 1.5 degrees of freedom (*heavy*), 16 locations with many duplicates, and a nearly degenerate line. Real data: UMAP embeddings [30] of all 70,000 MNIST digits [24] and all 70,000 Fashion-MNIST images [49] (15 neighbours, min_dist 0.1, fixed seed). Targets: an exact-fit near-square grid (*square*, last row partially usable when needed), the full rectangle with its few surplus cells (*free*), a rectangle with 10% surplus cells (*free10*), a 4:1 grid (*wide*), a disk, a ring, and a blocky, rectilinear letter S (*S mask*; three bars joined by vertical bars on alternating sides).

Comparators and adapters. Each adapter receives the same normalized float64 coordinates and cell centers and returns one cell per point. We check validity (injective, usable cells only) independently of every method.

- *Exact, dense*: SciPy `linear_sum_assignment` on the float64 cost matrix, the fastest of three dense solvers we tried (`lap.lapjv` and `lapjv` were 3–8× slower at $N = 2{,}500$).
- *Exact, certified sparse*: Section 5, started from ShuffleSnap's default (balanced) preset, whose time is included, or from bisection.
- *RasterFairy* [21]. The PyPI release 1.0.6 (2019) uses the removed NumPy alias `np.float`, and it orders points by *integer-truncated* coordinates. With grid-unit inputs this discards sub-cell order: excess was 193% without our remedy of multiplying inputs by 1000, and 82% with it (uniform, $N = 2{,}500$). RasterFairy's optional swap optimizer cannot run at the benchmark sizes: the PyPI copy is Python 2 code (a `SyntaxError` on Python 3), and the upstream copy, which does run on Python 3, allocates a dense $N \times N$ distance matrix (80 GB at 10^5 points). We test the upstream copy separately at $N = 2{,}500$ (Section 7.2, Table 10). The upstream GitHub master (July 2026) normalizes internally, sorts on floats, and solves leaf blocks of up to 200 points exactly with SciPy. It is much better (9% excess on the same instance), and it is our representative setting. Its per-leaf console printing is captured so that it does not inflate the timing. Surplus cells are removed at random by its `autoAdjustCount` (default strategy).
- *BSP-OT* [12] through its official Python binding `bspot` 1.2.0 [43]. Bijections use its matching function with Gaussian slicing, as in the upstream demo; surplus cells use its partial-matching function, which has no Gaussian option. We report 16 plans (the default) and 64 plans (as in the paper's figure), and sweep 1–1024 plans for time–quality curves. The binding always runs the cycle-merge step, whereas the upstream C++ wrapper disables it at $N \geq 500{,}000$. It exposes no random seed, so repeated calls return different assignments: on the 198 instances run with both 1 and 24 threads, the two costs never coincided and differed by -8% to $+13\%$, with a median difference of zero.
- *Bisection*: the recursive median split alone, an $O(N \log N)$ reference close to one BSP matching with axis-aligned cuts.

Neighbourhood measures. Displacement is not neighbourhood preservation, so we measure the latter separately on 5,000 sampled points: *kNN recall*, the fraction of each point's 10 nearest input neighbours that are among its 10 nearest grid neighbours (ties at the 10th grid distance

count), and *lattice precision*, the fraction of occupied 8-neighbour cells whose points are among the 8 nearest input neighbours.

7 Results

7.1 Correctness checks

Three properties were checked independently of the kernel:

- **Window solver.** Two unit tests cover it. The first compares against brute-force enumeration on random, tie-heavy and badly scaled matrices up to 5×6. The second compares against SciPy's `linear_sum_assignment` on square and rectangular matrices up to 144×144, including squared-distance matrices like the real use case. Both run for all five window solvers. A third test gives the geometric solver (Section 4.2) real window geometry, including far-away tight clusters, coincident points, elongated clusters, lattice-aligned ties and empty cells, and compares it against SciPy. On about 79,000 windows captured from real runs, it matched SciPy's optimum on every window.
- **Validity.** A NumPy validator, sharing no code with the kernel, checked that every output of every method gives each point its own usable cell. All 1,217 completed runs of the main benchmark were valid.
- **Monotonicity.** Each ShuffleSnap run records the exact cost after every phase. In all 540 ShuffleSnap runs of the main benchmark, this trace was monotone, and its final value matched the cost recomputed by the validator.

The test suite also checks that results do not depend on the thread count or on skipping unchanged windows, that the window tiling matches an independent reference implementation, and that the serial NumPy/SciPy reference of the whole algorithm (Appendix B) returns exactly the kernel's assignment for every preset on six kinds of target.

Small instances. On 600 small instances ($N \in \{100, 400, 900, 2500\}$; five distributions; exact-fit, 10%-surplus and disk targets; 10 seeds each), the certified solver agreed with SciPy's dense solver on every instance. Table 1 shows how often each method reached the optimum. ShuffleSnap is reliably exact only when its windows cover a large part of the grid: the quality preset's 12×12 window covers the whole 10×10 grid at $N = 100$. By $N = 2{,}500$ the default never reached the optimum, although its median excess was only 0.037%.

7.2 Quality and time at 10^5 points

Table 2 and Figure 2 summarize the held-out comparison at 10^5 points on one thread.

Table 1: Small instances: fraction of the 150 instances per size (five distributions, three target types, 10 seeds) on which the method reached the certified optimum (relative excess $< 10^{-9}$), and the median excess.

	SS fast	SS balanced	SS quality	BSP-OT 16	BSP-OT 64
$N = 100$	0.37 / 0.136%	0.83 / 0.000%	1.00 / 0.000%	0.00 / 9.2%	0.01 / 4.0%
$N = 400$	0.00 / 0.257%	0.17 / 0.064%	0.49 / 0.000%	0.00 / 8.1%	0.00 / 4.4%
$N = 900$	0.00 / 0.222%	0.00 / 0.070%	0.13 / 0.016%	0.00 / 9.0%	0.00 / 4.2%
$N = 2500$	0.00 / 0.122%	0.00 / 0.037%	0.00 / 0.011%	0.00 / 9.7%	0.00 / 4.9%

Table 2: Mean excess over the certified optimum (%) at $N = 100{,}000$, square exact-fit grid, instance seeds 1–3, one thread. Each ShuffleSnap preset is placed next to the BSP-OT setting that takes about as long. The last two columns give the time of the certified exact solver (seed 1) started from the default ShuffleSnap preset (time included) or from bisection. ^{t}: no certificate within the 1-hour limit; ^{n}: stopped after 50 pricing rounds without a certificate. The bottom row is the median time over all runs. Heavy tails are below 0.001% for every method in relative terms; Table 7 gives absolute excess.

distribution	SS fast	BSP 16	SS bal.	BSP 64	SS qual.	RF master	RF PyPI	Bisect.	exact (s)	exact, bis. (s)
uniform	2.58	44.8	0.84	32.1	0.44	44.8	61.6	100.3	16.5	67.9
Gaussian	<0.001	0.055	<0.001	0.040	<0.001	0.006	0.010	0.018	55.7	141
Gaussian mixture	0.013	5.61	0.002	3.74	<0.001	16.7	16.7	12.3	94.6	>3600^{t}
90/10 clusters	0.050	0.032	0.012	0.019	<0.001	1.16	1.16	0.57	214	1048
ring	0.005	1.16	0.001	0.95	<0.001	41.0	41.0	40.3	114	3117
spiral	0.022	9.96	0.004	8.06	0.001	27.3	27.4	34.4	120	3331^{n}
checkerboard	0.13	49.8	0.057	36.1	0.020	425.2	426.3	450.0	64.9	423
heavy tails	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	261	320
duplicates	0.15	12.3	0.038	5.44	0.013	49.0	49.0	39.3	1364	>3600^{t}
near-line	0.010	0.008	0.002	0.002	<0.001	19.4	19.4	19.1	253	>3600^{t}
median time (s)	0.77	0.71	2.33	2.85	6.67	3.00	82.2	0.034	117	

ShuffleSnap. The default (balanced) preset is within 0.06% of the optimum on nine of the ten distributions, and within 0.005% on six of them. It needs a median of 2.3 s on one thread. The exception is uniform points (0.84%), where the long loops of Section 7.8 remain. The presets trade time for quality on every distribution. Fast (median 0.77 s) is within 0.16% on the same nine distributions; quality (median 6.7 s) lowers the excess on uniform points to 0.44%.

At equal time. The fast preset takes about as long as BSP-OT with 16 plans (0.77 s against 0.71 s), and the default a little less than BSP-OT with 64 plans (2.3 s against 2.9 s). At these matched times ShuffleSnap is closer to the optimum on eight and on nine of the ten distributions. The fast preset is at 2.6% on uniform points, 0.013% on the mixture and 0.13% on the checkerboard, where BSP-OT with 16 plans is at 45%, 5.6% and 50%. The default is at 0.84%, 0.002% and 0.057% there, against 32%, 3.7% and 36% for 64 plans. BSP-OT is ahead only on two inputs whose structure suits its partitions: the near-line (0.008% against 0.010% for fast; 0.0021% against 0.0022% for the default) and the 90/10 clusters (0.032% against 0.050% for fast).

RasterFairy (upstream) takes about 3 s. Its recursive slicing, however, costs 17–425% on seven of the ten distributions. It does well on two: the Gaussian (0.006%) and the 90/10 clusters (1.2%). The PyPI release, with pre-scaled inputs, gives similar or worse costs (e.g. 62% vs. 45% on uniform points) and takes 80 s at this size. Its targets at 50k and 100k points have a partially filled last row, which sends it into its much slower mask code path; at $N = 10^4$, a full 100×100 square, it needs 0.03 s.

RasterFairy's swap optimizer needs a dense $N \times N$ distance matrix, so we ran it only at $N = 2{,}500$ (49 MB), on the same ten distributions (Table 10). Started from RasterFairy's layout, it keeps only improvements, but each iteration recomputes the full cost in Python. A million iterations take 63 s and remove a median of 26% of RasterFairy's excess (7–91%; most on the near-line, least on Gaussian and uniform points). Ten million iterations (10 minutes) remove a median of 43% (13–96%). Even then the excess is 4–320 times that of the fast preset, which takes 0.012 s, and below that of BSP-OT with 64 plans on only four distributions. The dense SciPy solver finds the

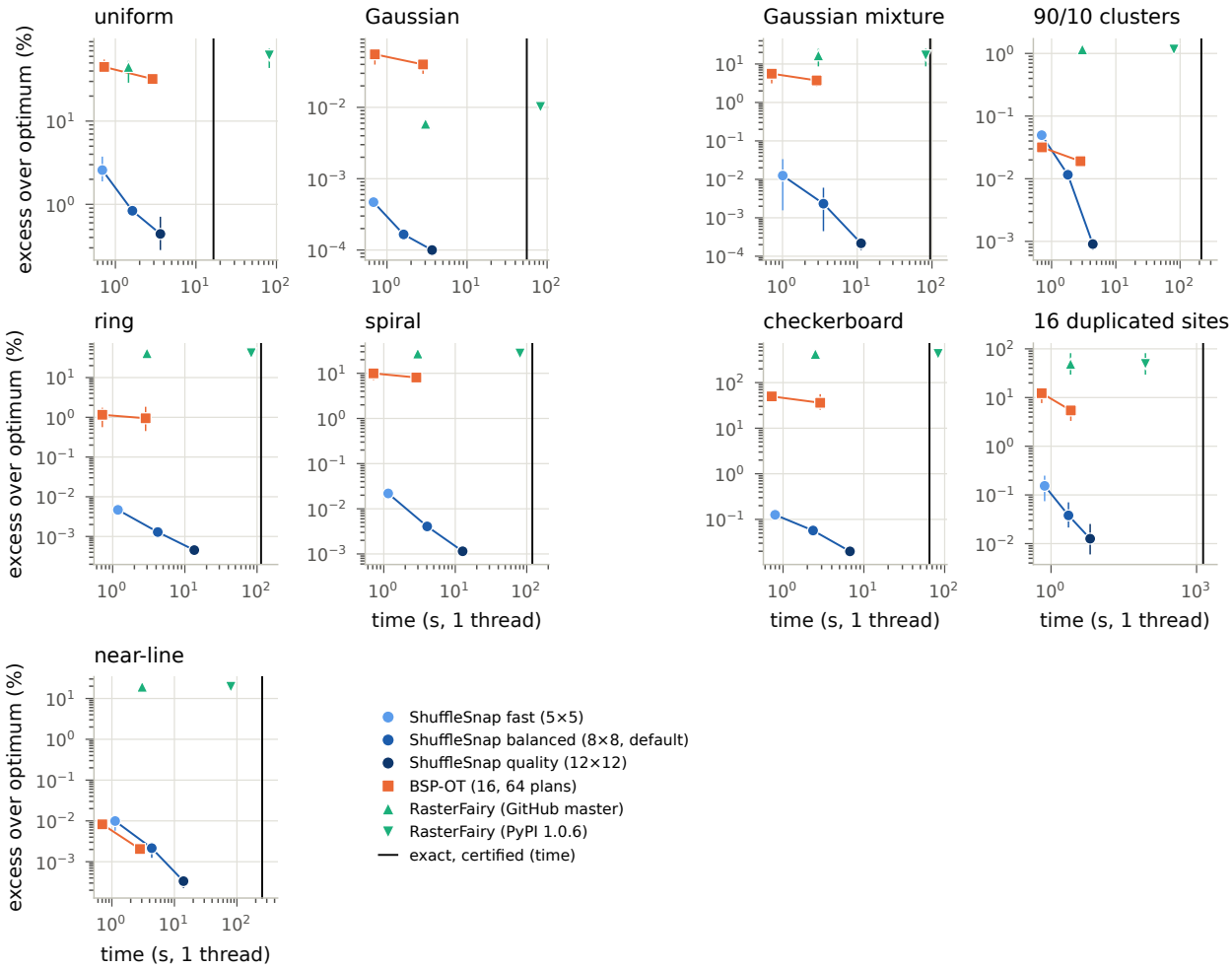


Figure 2: Excess over the certified optimum against single-thread time at $N = 100{,}000$ (mean over three held-out instances; bars span min–max). The three ShuffleSnap presets form the blue curve, from fast (5×5) to quality (12×12). BSP-OT at 16 and 64 plans is joined by an orange line. The black vertical line is the time of the certified exact solver. Heavy tails are omitted (all methods <0.001%).

optimum in 2.7 s. Started from a random permutation, as in upstream's demo, a million iterations leave 0.4–6300% excess.

With all cores. On 24 hardware threads, ShuffleSnap ran 7–14× faster than on one and returned bit-identical assignments in all 270 paired runs. At 10^5 points the median time is 0.09 s for fast, 0.19 s for the default and 0.53 s for quality. BSP-OT gained 4–7× (0.13 s with 16 plans, 0.41 s with 64); its plans are computed in parallel, but the cycle-merging pass that combines them is largely sequential. At 24 threads the fast preset is therefore faster than BSP-OT with 16 plans (0.09 s against 0.13 s), and the default is faster than BSP-OT with 64 plans (0.19 s against 0.41 s).

Exact solutions. Started from ShuffleSnap's default preset, the certified solver finished every instance, in 17 s (uniform points) to 23 min (16 duplicated sites), with a median of 117 s. That is 10–620 times the time of the default preset (median 32 times). Started from bisection instead, it failed on four instances (mixture, duplicates, near-line, spiral): no certificate within an hour or 50 pricing rounds. On the six instances where it did certify, it was 1.2–27 times slower. The start

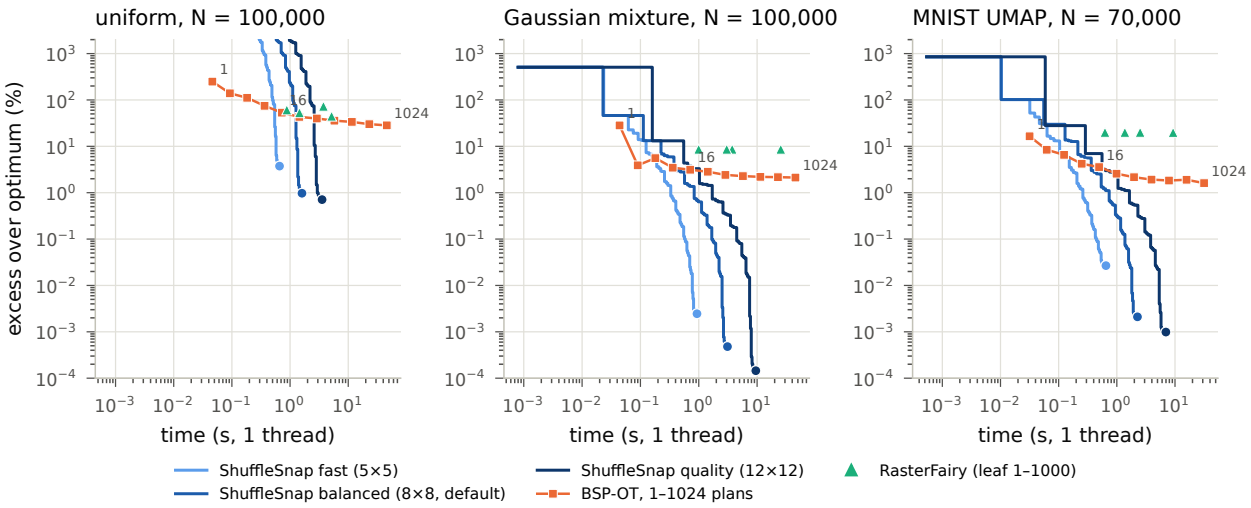


Figure 3: Excess over the certified optimum against elapsed time, one thread. ShuffleSnap's step curves record the exact cost after every phase, so each shows the result that a time budget ending at that moment would return. BSP-OT is run separately with 1, 2, 4, …, 1024 plans (labels at 1, 16 and 1024). RasterFairy upstream is shown with exact leaf blocks of 1, 50, 200 and 1000 points. Instance seed 1. The same plot with 24 threads is in the appendix (Figure 13).

matters because the candidate edges come from it: ShuffleSnap's layout is close enough to optimal that few pricing rounds are needed. A dense exact solver is not an option at this size. Already at $N = 10^4$, SciPy needed 11–352 s (Table 8 in the appendix), whereas the certified sparse solver needed 0.5–6.9 s.

7.3 Quality at a matched time budget

Figure 3 compares the methods at matched time budgets. BSP-OT improves slowly with more plans and levels off far from the optimum. From 1 to 16 to 1024 plans, its excess drops from 249% to 53% to 28% on uniform points, from 28% to 3.1% to 2.1% on the mixture, and from 16% to 3.6% to 1.6% on MNIST. The 1024-plan runs take 31–46 s on one thread. The fast ShuffleSnap preset passes that best BSP-OT result after 0.2–0.5 s and the default after 0.5–1.3 s. They finish at 3.7%, 0.002% and 0.027% (fast) and at 0.97%, 0.0005% and 0.002% (default). BSP-OT with 1–8 plans is the better choice only for budgets below 0.12–0.36 s on one thread, when ShuffleSnap has finished only its coarsest levels. With 24 threads, the fast preset is ahead at every budget for which BSP-OT returns a result. Larger windows are slower not only per round but also to any cost a smaller window reaches: the quality preset needed 2.8–7.4 s to reach the cost at which the fast preset stops after 0.7–0.9 s, and the default 1.4–2.6 s. A larger window pays off only at costs that a smaller one never reaches. RasterFairy's leaf size hardly matters except on uniform points (73% with no exact leaves, 45% with 1000-point leaves).

7.4 Scaling in time and memory

Between 10^4 and 10^6 points, ShuffleSnap, BSP-OT and RasterFairy all scale close to linearly. Their fitted exponents of time in N are 1.15–1.18, 1.08 and about 1.0 (Figure 4). The time match with BSP-OT holds at 10^6 uniform points: the fast preset takes 9.8 s on one core and BSP-OT with 16 plans 9.4 s. The default takes 21 s; with 24 threads, fast and default take 0.9 s and 1.5 s, and

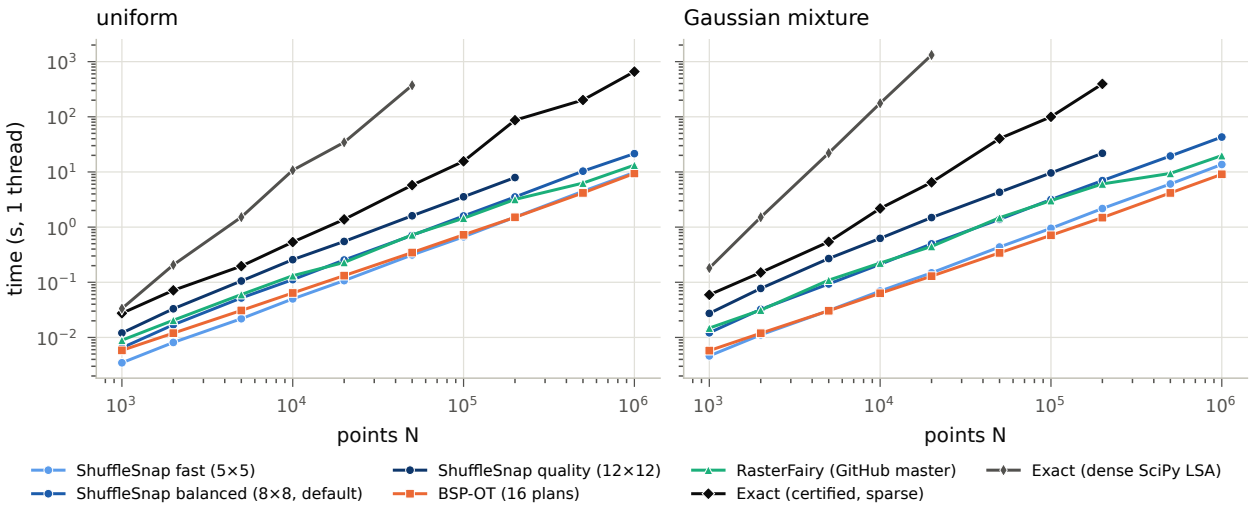


Figure 4: Time against N on one thread (instance seed 1). Dense SciPy was run up to the largest size we considered sensible, 50k uniform points (6.2 min, 23 GiB) and 20k mixture points (22 min). The certified solver was run up to 10^6 uniform and 2×10^5 mixture points. The quality preset was run up to 2×10^5 points.

Table 3: Peak extra resident memory (MiB) during one call, measured on a heap trimmed right before the call (`malloc_trim`), one thread, instance seed 1. Without trimming, small allocations can reuse pages freed while the instance was generated and go unseen.

	uniform			Gaussian mixture		
method	10^4	10^5	10^6	10^4	10^5	10^6
ShuffleSnap fast	0.6	5.4	52.2	0.6	5.5	52.2
ShuffleSnap balanced	0.6	5.4	52.2	0.6	5.5	52.2
ShuffleSnap quality	0.6	5.4	52.2	0.6	5.5	52.2
BSP-OT, 16 plans	2.2	24.2	243	2.4	24.3	244
RasterFairy master	6.2	63.4	647	6.2	62.9	647
exact, certified sparse	49.5	437	–	76.5	772	–
exact, dense SciPy	1563	–	–	1564	–	–
bisection only	0.1	1.2	11.4	0.1	1.1	13.3

BSP-OT 2 s. On the mixture, 10^6 points take 14 s (fast) and 43 s (default) on one thread, or 1.0 s and 2.7 s on 24 threads. RasterFairy takes 13–20 s on one thread.

Exact solving scales worse but reaches further than we expected. The certified solver's time grew as $N^{1.6}$ on uniform points and $N^{1.7}$ on the mixture. It certified 10^6 uniform points in 11 min using 4.2 GiB, and 2×10^5 mixture points in 6.6 min. Its memory, 4–8 kB per point, is set by the candidate edges and the flow solver. Dense SciPy grew as $N^{2.2}$ (uniform) to $N^{2.9}$ (mixture) and needs $8N^2$ bytes. It took 6.2 min and 23 GiB at 50k uniform points, and 22 min at 20k mixture points. ShuffleSnap needs $O(N)$ working memory: an occupancy array, a position array, a last-change stamp per cell, the window lists of the current level, and the input checks' temporary arrays. Measured, this is 55–58 bytes per point at 10^5 and 10^6 points, or 52 MiB at 10^6 (Table 3). The comparators need 4–12× more: BSP-OT about 250 and RasterFairy about 670 bytes per point.

At 10^6 uniform points, the quality ranking is unchanged. Measured against the certified optimum of that instance, the excess is 3.3% for fast, 1.3% for the default and 0.58% for quality, against 74% for BSP-OT (16 plans) and 48% for RasterFairy. On uniform points the default's excess grows

with N: 0.11% at 10^3, 0.97% at 10^5 and 1.3% at 10^6 (one instance each).

A billion points. To see how far the method reaches beyond the benchmark, we solved the largest instance of the earlier ShuffleSnap work: 1,000,014,129 points of a 2D random walk (the "meander"), normalized to the unit square, on a $31{,}623 \times 31{,}623$ grid. We used the same random-permutation start, the default preset and 24 threads, on the same workstation (Figure 5). A dense cost matrix would take 8 EB. Setting up the start and the occupancy took 32 s, and memory peaked at 53 GiB, 57 bytes per point: coordinates and cell centers take 16 bytes each, change stamps 8, positions, occupancy and the grid index 4 each, and the window lists about 5. The kernel accepts any mask; for a full rectangle, the centers and the index could be computed instead of stored, which would save 20 of the 57 bytes. The walk is so concentrated that the result moves the average point by about 6,900 cells (root mean square).

The default preset reached its fixed point after 1 h 43 min (stride 1 began at 1 h 30 min, and 78 stride-1 rounds followed). The coarse levels took about 4 minutes each, because at this size nearly every window changes at every level and none can be skipped. We then ran the quality preset's last four levels (12×12 windows, strides 4 to 1), which converged after 4.1 h in total and lowered the cost by only 1.2×10^{-6}%. Progress was front-loaded, as at smaller sizes: the cost was within 1% of the final one after 11 minutes, 0.1% after 22, 0.01% after 34 and 0.001% after 46, and after one hour it was only 0.0001% above it.

No optimum is known for this instance, so we priced other assignments of the same instance on the same frame. The earlier implementation (6×6 windows, halving strides, 24 threads) ran for six hours without converging. It reached stride 1 after 87 minutes and ended 0.0017% above our final cost, a level our run passed after 44 minutes. A memory-lean single-plan port of BSP-OT, written for the earlier work because the official multi-plan code does not fit in memory at this size, took 392 s on one thread and is 8.2% above. This is one instance with no optimality certificate. It shows that the method runs to convergence at 10^9 points on one workstation, not how close it comes to the optimum there.

7.5 Real embeddings: displacement versus neighbourhoods

Figure 6 and Table 4 compare the methods on the full MNIST and Fashion-MNIST embeddings. The default preset is within 0.002% of the certified optimum on both datasets, and the fast preset within 0.031%. At about the same time as BSP-OT with 16 plans (0.66 s against 0.49 s on one core), the fast preset is at 0.031% and 0.008%, where BSP-OT is at 3.7% and 9.2%. In every panel their colour fields match the optimum's.

The two measures disagree on BSP-OT. With 64 plans, its displacement is only 2.4% (MNIST) and 6.7% (Fashion) above optimal, yet both neighbourhood measures fall to about half: kNN recall 0.31–0.33 against 0.61–0.65 for the optimum, and lattice precision 0.25–0.28 against 0.55–0.58. Averaged over plans, the merged BSP partitions are close to the transport map, but they shuffle points locally. RasterFairy shows the opposite pattern: its displacement is 20–25% above optimal, yet its neighbourhood measures are close to the optimum's. On Fashion-MNIST its kNN recall (0.611) even exceeds the optimum's (0.606), because local order survives inside each slice and breaks only along the seams. ShuffleSnap's measures match the optimum's to within 0.003. Compared cell by cell with the optimum (row 4, the figure's runs), ShuffleSnap leaves 74% (MNIST) and 69% (Fashion) of the points on their optimal cell and 93% and 91% within one cell, while BSP-OT's and RasterFairy's median points are 8 and 19 cells from theirs on both embeddings. Even the optimum has lattice precision well below 1: an optimal transport map from clustered data to

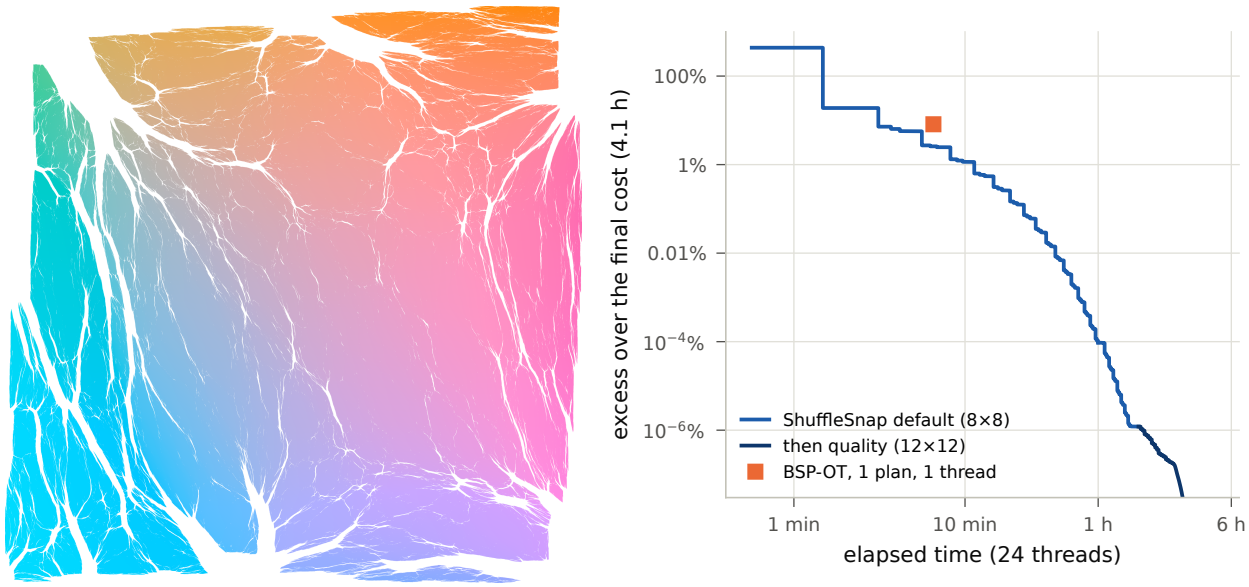


Figure 5: A billion points. Left: every point drawn 75% of the way from its input position to its assigned cell, one pixel per point, colored by input position and averaged over $10{,}000^2$ pixels (shown downsampled). White seams open where neighbouring cells hold points from distant sources; they follow the walk's folds. Right: cost after every phase as excess over the final cost: the default preset until its fixed point, then the quality preset's last four levels. A single-plan BSP-OT port is shown for comparison.

Table 4: Real embeddings, all 70,000 points (square exact-fit grid). Excess over the certified optimum and neighbourhood measures are means over three method seeds; times are medians on one thread (1T) and on 24 threads (24T, methods that use threads only). Memory is the rise of peak resident memory during the call. kNN recall: fraction of each point's 10 nearest input neighbours among its 10 nearest grid neighbours. Lattice precision: fraction of occupied 8-neighbour cells holding one of the point's 8 nearest input neighbours.

	method	excess (%)	mean sq.	time 1T (s)	time 24T (s)	mem (MiB)	kNN recall	lattice prec.
MNIST UMAP	exact (certified)	0	2284.7	82.0	–	562	0.645	0.581
	ShuffleSnap fast	0.031	2285.4	0.66	0.064	2	0.644	0.578
	ShuffleSnap balanced	0.002	2284.7	2.27	0.16	2	0.645	0.580
	ShuffleSnap quality	<0.001	2284.7	7.03	0.54	2	0.644	0.580
	BSP-OT, 16 plans	3.73	2370.0	0.49	0.088	16	0.305	0.255
	BSP-OT, 64 plans	2.42	2340.0	1.98	0.28	42	0.305	0.249
	RasterFairy master	20.0	2742.4	1.38	–	42	0.631	0.551
	RasterFairy PyPI	44.5	3301.0	11.2	–	44	0.602	0.534
	bisection only	34.4	3071.1	0.023	–	0	0.569	0.498
Fashion UMAP	exact (certified)	0	1802.6	76.0	–	566	0.606	0.548
	ShuffleSnap fast	0.008	1802.7	0.67	0.056	2	0.606	0.548
	ShuffleSnap balanced	0.001	1802.6	2.31	0.17	2	0.607	0.547
	ShuffleSnap quality	<0.001	1802.6	7.23	0.56	2	0.607	0.547
	BSP-OT, 16 plans	9.15	1967.5	0.50	0.088	16	0.295	0.245
	BSP-OT, 64 plans	6.73	1923.8	1.98	0.28	42	0.334	0.275
	RasterFairy master	24.6	2245.3	1.38	–	42	0.611	0.532
	RasterFairy PyPI	25.6	2264.5	11.1	–	44	0.594	0.526
	bisection only	23.0	2217.7	0.023	–	0	0.544	0.477

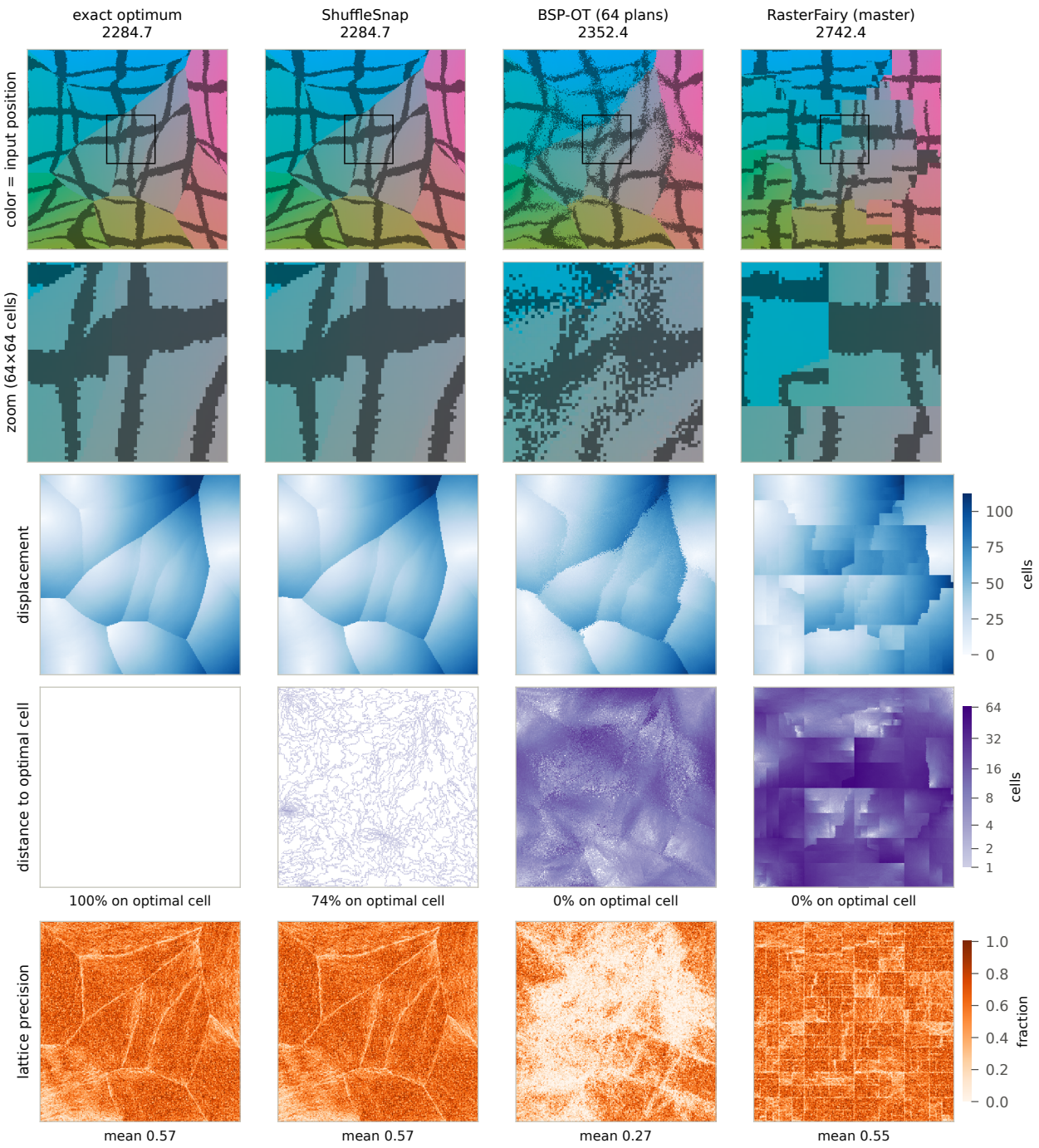


Figure 6: All 70,000 MNIST digits, UMAP layout snapped to a 265×265 grid. Each point has the same color in every panel, set by its position in the input layout, and the dark bands are graph-paper lines drawn on the input, so bent lines show how a method deforms the layout, and color jumps or broken lines show where it tears it. The number under each method name is the mean squared displacement in cells^2. Row 2 enlarges the 64×64 box. Row 3 shows each point's displacement. Row 4 shows the distance from each point's cell to its cell in the certified optimum (logarithmic scale; white means the optimal cell, so the optimum's own panel is blank). Row 5 shows lattice precision: the fraction of a point's occupied 8-neighbour cells whose points are among its 8 nearest input neighbours (higher is better). ShuffleSnap is visually indistinguishable from the certified optimum: 74% of its points sit on their optimal cell, and its few errors form the thin loops of one-cell shifts of Section 7.8. BSP-OT is within 3% of the optimal displacement, yet its median point is 8 cells from its optimal cell, and it halves lattice precision, because it scrambles local order (frayed lines and speckled boundaries in rows 1–2). RasterFairy's recursive slicing breaks the lines at rectangular seams.

Table 5: Excess over the certified optimum (%) on non-square targets, $N = 20{,}000$, mean of instance seeds 1–3 (all 24 exact runs certified). "Surplus (<1 row)" is the full $W \times H$ rectangle holding N points, with fewer than W empty cells; "10% surplus" has $1.1N$ cells.

target	points	SS fast	BSP 16	SS bal.	BSP 64	SS qual.	RF master
surplus (<1 row)	uniform	6.31	90.3	3.10	85.0	1.37	56.6
surplus (<1 row)	Gaussian mixture	0.033	8.41	0.013	7.95	0.010	17.2
surplus (<1 row)	spiral	0.099	19.9	0.006	11.5	0.003	32.4
surplus (<1 row)	MNIST UMAP	0.082	16.5	0.020	5.29	0.011	43.6
10% surplus	uniform	3.81	108.9	0.91	58.7	0.25	350.5
10% surplus	Gaussian mixture	0.43	30.5	0.055	17.6	0.012	71.6
10% surplus	spiral	0.61	26.2	0.071	19.3	0.012	157.9
10% surplus	MNIST UMAP	0.077	29.4	0.007	22.8	0.003	146.2
4:1 rectangle	uniform	2.64	32.3	1.34	23.9	0.45	26.4
4:1 rectangle	Gaussian mixture	0.23	1.44	0.024	0.81	0.004	6.70
4:1 rectangle	spiral	0.34	4.33	0.067	2.34	0.022	15.4
4:1 rectangle	MNIST UMAP	0.18	2.07	0.014	1.28	0.003	11.6
disk	uniform	0.034	8.64	0.014	6.74	0.006	11.4
disk	Gaussian mixture	0.016	5.43	0.003	4.27	<0.001	37.0
disk	spiral	0.024	5.19	0.007	2.48	0.003	23.6
disk	MNIST UMAP	0.022	4.49	0.006	2.77	0.002	20.3
ring	uniform	0.067	9.28	0.028	6.73	0.006	53.6
ring	Gaussian mixture	0.11	6.75	0.048	4.81	0.018	35.9
ring	spiral	0.048	7.73	0.014	3.83	0.016	47.9
ring	MNIST UMAP	0.14	4.77	0.070	3.11	0.025	43.2
S mask	uniform	16.6	12.1	1.43	9.30	0.88	60.8
S mask	Gaussian mixture	0.63	4.11	0.057	2.99	0.036	32.4
S mask	spiral	3.27	12.5	0.26	4.79	0.19	56.8
S mask	MNIST UMAP	1.55	6.23	0.12	4.53	0.093	19.0

a uniform grid tears the layout along cluster boundaries (the white lines in row 5). Minimizing displacement and preserving neighbourhoods are different objectives, and neither implies the other.

7.6 Masks, surplus cells and prime counts

Table 5 and Figure 7 cover non-square targets.

- **Convex masks.** On disks and rings, the default preset stays within 0.07% of the optimum, fast within 0.14% and quality within 0.025%. BSP-OT is 2.5–9.3% above it and RasterFairy 11–54%.
- **The S-shaped mask** (the blocky, rectilinear S of the earlier ShuffleSnap work: three horizontal bars joined by vertical bars on alternating sides) is where the schedule matters most. Coarse strides can route mass across its bays (Figure 9c). The default reaches 1.4% with uniform points and 0.26% with the spiral, against 9.3% and 4.8% for BSP-OT with 64 plans, and the quality preset 0.88% and 0.19%. The fast preset is at 3.3% with the spiral, against 12.5% for BSP-OT with 16 plans, but with uniform points it is the one mask case where BSP-OT wins at equal time: 17% against 12%.

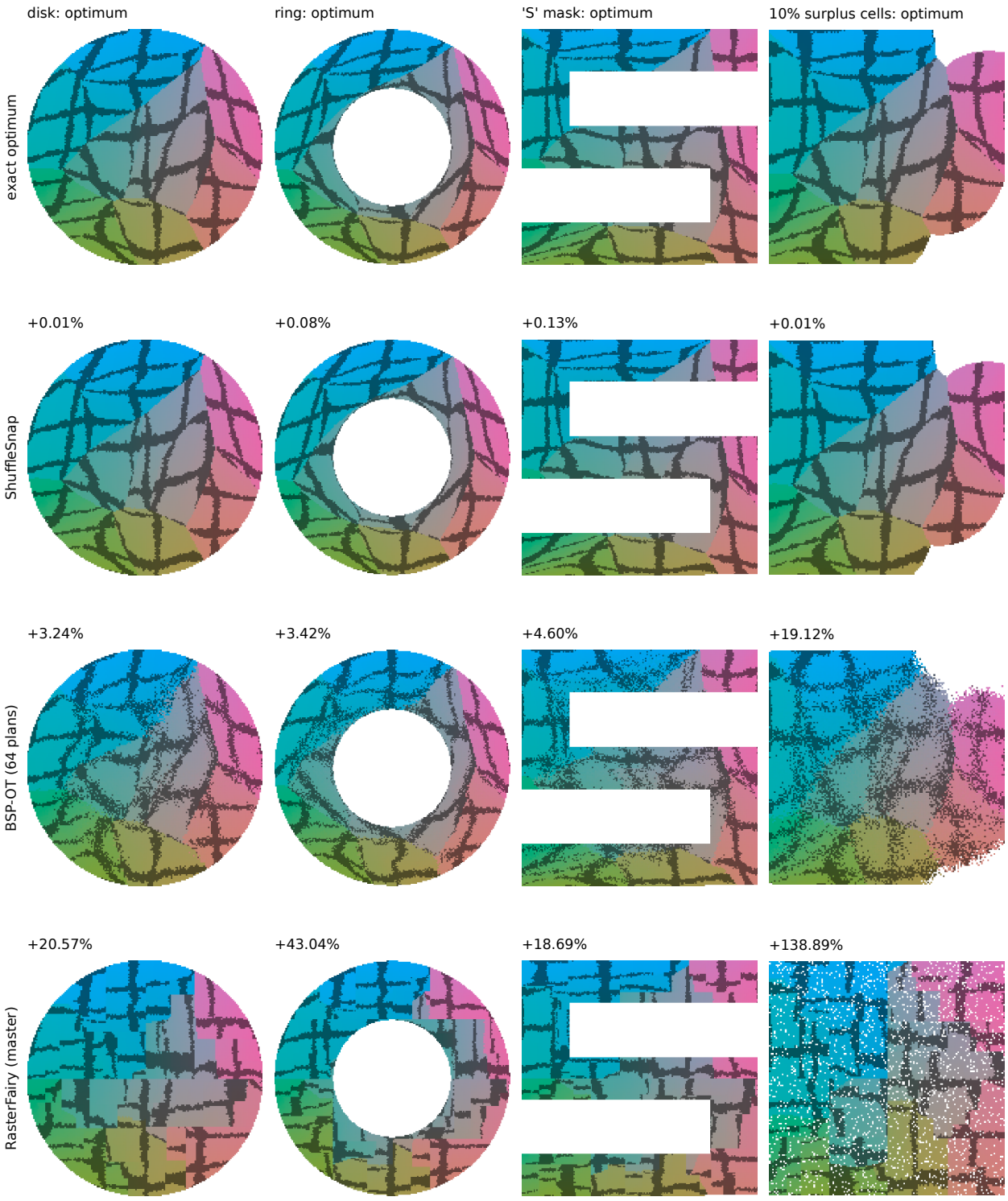


Figure 7: A 20,000-point subset of the MNIST UMAP embedding on a disk, a ring, a rectilinear S-shaped mask, and a rectangle with 10% surplus cells (instance seed 1; colors and graph-paper lines encode input position as in Figure 6; white = empty or masked). Numbers are excess over the certified optimum. With surplus cells, the optimum leaves one contiguous region empty, far from the data, and ShuffleSnap (default preset) finds the same region. BSP-OT's partial matching leaves ragged holes along the boundary, and RasterFairy removes surplus cells at random before slicing.

Table 6: Prime point counts, excess over the certified optimum (%), mean of instance seeds 1–3 (24 threads; all 8 timed exact runs certified). The *exact fit* target is a near-square grid whose last row is partially usable, so every cell is filled. The *full rectangle* has a few surplus cells (177 for N = 49,999, 181 for N = 99,991), and the solver must also choose which cells stay empty.

N	target	points	SS fast	SS bal.	SS qual.	BSP 16	RF master
49,999	exact fit	uniform	1.93	0.85	0.35	36.8	31.7
49,999	exact fit	Gaussian mixture	0.013	0.002	<0.001	5.32	36.9
49,999	full rectangle	uniform	10.9	5.72	2.96	146.6	81.9
49,999	full rectangle	Gaussian mixture	0.096	0.079	0.016	12.0	39.1
99,991	exact fit	uniform	2.80	1.19	0.43	56.1	47.4
99,991	exact fit	Gaussian mixture	0.014	0.003	<0.001	4.77	16.0
99,991	full rectangle	uniform	10.3	5.14	3.63	152.4	70.7
99,991	full rectangle	Gaussian mixture	0.076	0.021	0.019	8.21	17.1

- **Surplus cells.** When the solver must also choose the empty cells, uniform points with less than one row of surplus are the hardest case: 3.1% for the default, 6.3% for fast and 1.4% for quality. With 10% surplus the default stays within 0.91% on every input. Surplus is where the comparators suffer most. With 10% surplus, BSP-OT's partial matching is 18–109% above the optimum, and RasterFairy, which picks the empty cells at random, is 72–350% above it. With less than one row of surplus, they are at 5–90% and 17–57%.

Prime counts. A prime number of points never fills a rectangle. Table 6 compares the two ways of handling this. With an exact-fit grid, the results match the square case: 0.85–1.2% for the default on uniform points, and 0.002–0.003% on the mixture. Leaving the solver to pick fewer than 200 empty cells in a full rectangle is much harder on uniform points: the default reaches 5.1–5.7% there, against 0.85–1.2% with the exact fit; fast reaches 10.3–10.9% and quality 3.0–3.6%. BSP-OT's partial matching is worse still, at 147–152%. For layouts where the empty cells' position does not matter, an exact-fit mask is therefore both simpler and better handled by every method.

7.7 Ablations on held-out instances

Figure 8 varies one design choice at a time, starting from the default, on instances that were not used for any design decision.

- **Window side** is the main quality knob. With $\sqrt{2}$ strides, uniform excess falls from 3.8% (4×4) to 0.84% (8×8), 0.44% (12×12), 0.19% (16×16) and 0.09% (24×24). Time rises from 0.06 s to 1.6 s on uniform points and from 0.06 s to 9.3 s on the mixture. With halving strides the whole curve lies higher (e.g. 1.5% at 8×8).
- **Stride ratio**. $\sqrt{2}$ is the best ratio on uniform points and MNIST. On the mixture, 1.68 and 1.19 are better (0.0017% and 0.0020% against 0.0023%). Ratios of 2 and 3 are worse on all three inputs (uniform: 1.5% and 2.4% against 0.84%).
- **Coarse levels are essential**. Capping the first stride at 16, 8, 4, 2 or 1 (the last means stride-1 windows only, from the random start) made results up to 90 times worse and up to 45 times slower. On uniform points the default reaches 0.84% in 0.12 s; capped at 8 it reaches 9.6% in 0.21 s, and with stride 1 only 21% in 0.47 s. Without long-range moves, stride-1 windows need many rounds to repair a random start, and they stall far from the optimum.

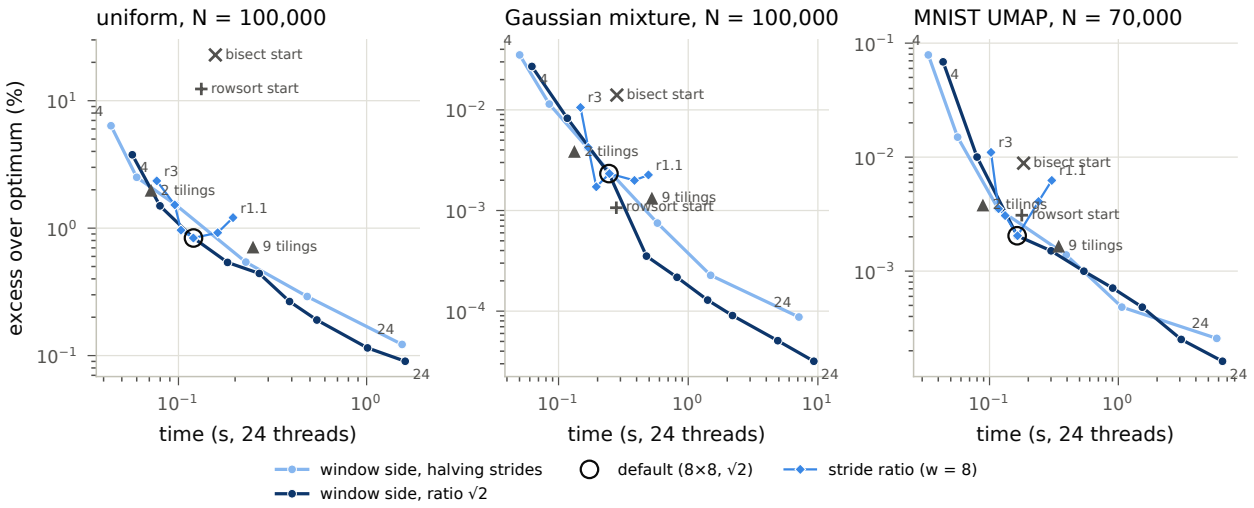


Figure 8: Held-out ablations. Each panel shows mean excess over the certified optimum against median time (24 threads, three instance seeds). The circle marks the default (8×8 windows, stride ratio $\sqrt{2}$). Lines vary an ordered setting: the window side with $\sqrt{2}$ strides (dark) or halving strides (light), labelled at 4 and 24; and the stride ratio at $w = 8$ (diamonds; r3 and r1.1 mark the extremes). Markers show categorical changes to the default: bisection or row-sort starts, and 2 or 9 shifted tilings per round instead of 4. A single tiling is off the chart on uniform points (104%). Solver choice and skipping unchanged windows do not change quality; they are listed with all other numbers in Table 11.

- **Start**. A random start beats bisection on all three inputs (uniform: 0.84% against 22.9%) and row sorting on uniform points and MNIST (12.4% and 0.0031% against 0.84% and 0.0020%). Bisection's straight seams are what windows cannot undo. On the mixture, a row-sorted start was better (0.0011% against 0.0023%).
- **Shifted tilings are essential**. With a single fixed tiling, points can never cross a window border (uniform: 104%). Two diagonal shifts give 2.0%, the default four 0.84%, and nine 0.70% at twice the time. A second round per level gives 0.70% at 1.4 times the time, less than a larger window gives for the same time (10×10: 0.54% at 1.5 times).
- **Window solver**. All five solvers are exact, so the results are identical up to ties. Relative to the Hungarian method with a greedy start, the geometric solver (Section 4.2) is 2.0–2.4× faster on uniform points and 5.0–5.4× faster on the mixture and MNIST. The auction with an exact finish is 2.1–3.4× faster on the clustered inputs but 1.04–1.3× slower on uniform points. The plain Hungarian method is up to 1.2× slower, and Jonker–Volgenant 3–12× slower.
- **Skipping unchanged windows** leaves results unchanged and saves 22–24% of the time.
- **Post-processing does not pay.** Three extra cycles of stride-2 and stride-1 rounds after convergence changed nothing on seven of the nine instances and removed at most 5% of the (already tiny) excess on the other two, both MNIST. Merging four independent default runs, keeping the cheaper of the two assignments on every connected component of their union (the rule BSP-OT uses to merge plans), lowered the uniform excess from 0.84% to 0.56% at 4.1 times the time. The quality preset does better (0.44%) at 2.1 times the time.

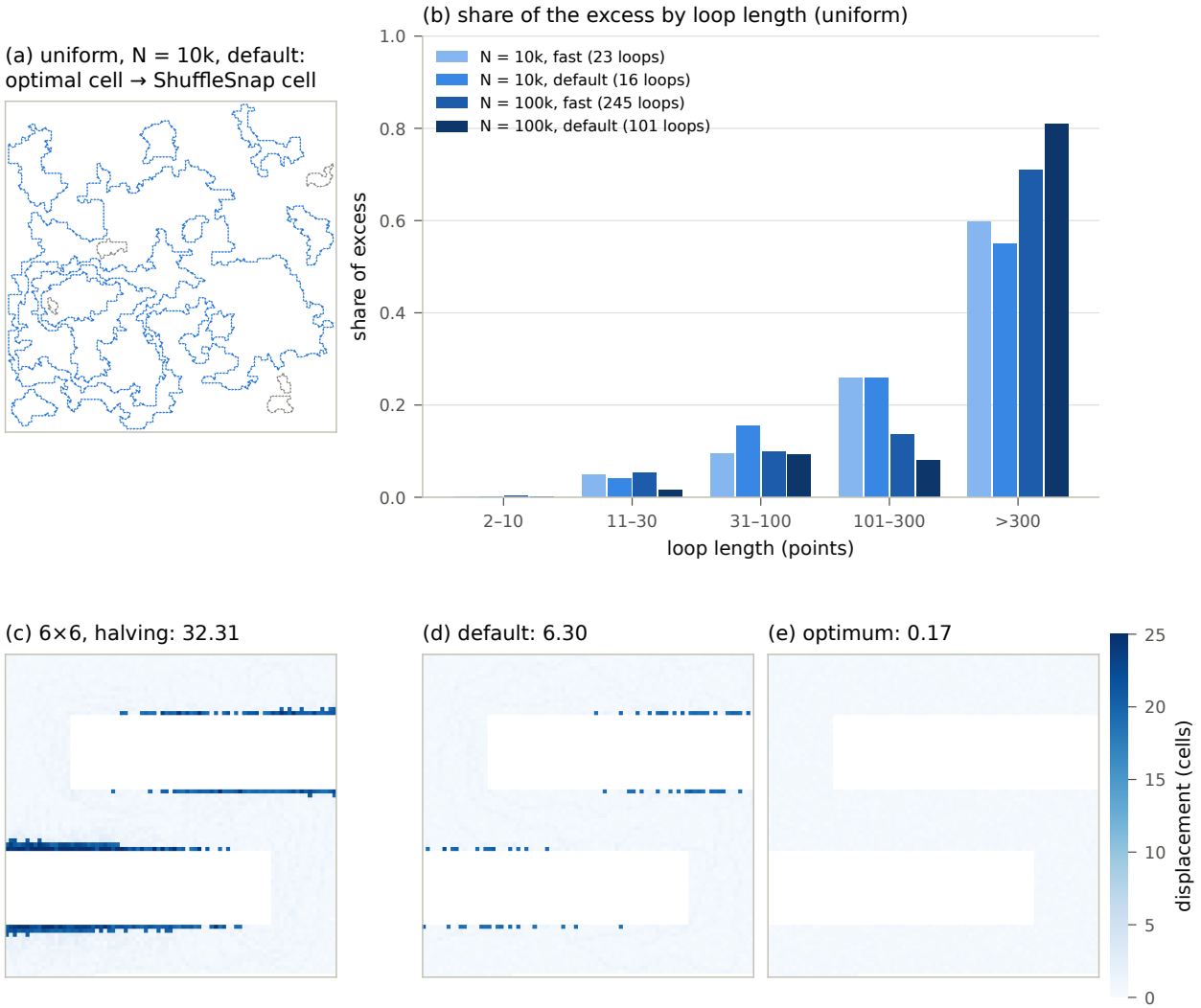


Figure 9: Where ShuffleSnap loses to the optimum (development instances, shown for mechanism rather than as claims). (a) 10,000 uniform points, default preset. Each arrow goes from a point's optimal cell to its ShuffleSnap cell. The two layouts differ by closed loops of one-cell moves; loops longer than 30 points are blue. (b) Share of the excess carried by loops of each length, for the fast and default presets: at 10^5 points, about 80% of it sits in loops of more than 300 points. (c–e) 4,000 points drawn uniformly inside the rectilinear S-shaped mask, so that the optimum moves every point by less than a cell. With 6×6 windows and halving strides, points are routed across the concave bays and end up in long lines along their edges; the default preset leaves fewer such lines. Numbers are mean squared displacement.

7.8 Anatomy of the remaining error

Why does a method that solves every window exactly still miss the optimum on uniform data, by 0.84% at 10^5 points? Compare a ShuffleSnap layout with the certified optimum. The permutation that turns one into the other splits into cycles (Figure 9a). Almost all the excess sits in a handful of very long cycles, in which every point is shifted by about one cell, so the layout is locally rotated along a closed curve (Figure 9b). Such a loop is invisible to every window:

- Inside a stride-1 window, the part of a loop is a run of points shifted consistently by one cell, which is already the best arrangement of that occupant set. Undoing the loop requires moving one point out at one end of the run and one in at the other end.
- Coarser windows cannot help either. A one-cell shift moves a point to another residue class, and a strided window only permutes points within one class.

Indeed, three extra cycles of stride-2 and stride-1 rounds after convergence changed nothing on held-out uniform instances.

A constructed fixed point. The argument can be checked directly. Put the points exactly on the cell centers of a 64×48 grid, so that the optimum is the identity with cost 0. Then rotate an axis-aligned ring of cells by one step: the point of each ring cell moves to the next ring cell, so the cost equals the number L of ring cells. We ran the complete $\sqrt{2}$ schedule from this layout, with every stride and every tiling, followed by stride-1 rounds to convergence. For windows of side $w \in \{5, 6, 8, 12, 16\}$, a ring larger than the window was never touched (0 points moved), so the result has cost L against an optimum of 0. A ring of side at most $w/2 + 1$ always lies inside one of the four shifted tilings and was repaired. A 6×6 ring survived 6×6 windows when it straddled the borders of all four tilings. Relative excess over the optimum is therefore unbounded for this method, and the package's test suite includes this configuration as a documented limitation.

On real inputs, larger windows remove the shorter loops, and the $\sqrt{2}$ schedule creates fewer loops in the first place, but the long loops remain: at 10^5 uniform points, loops of more than 300 points carry about 80% of the default's excess (Figure 9b). Their number and total length grow with N, which is why the default's excess on uniform points grows from 0.1% at 10^3 points to 0.84% at 10^5 and 1.3% at 10^6 (Section 7.4).

Masks make the same mechanism visible at a larger scale (Figure 9c–e). At a coarse stride, a window's cells can lie on both sides of a concave bay, so mass gets routed across the bay. By the time the stride is 1, fixing the routing would require moving a whole arm's worth of points along the arm, which no window can do. On points drawn uniformly inside the S, where the optimum costs 0.17 cells^2 per point, 6×6 windows with halving strides ended at 32 cells^2 per point, with the misrouted points lined up along the bays. The default's larger windows and finer $\sqrt{2}$ schedule revisit intermediate scales and reduce the error (6.3 cells^2 per point) but do not remove it; the held-out numbers are in Section 7.6.

A mask can also cut exchanges off entirely. Two usable cells trade occupants only if some scheduled window contains both. On a one-row mask whose only usable columns are 0 and 9, stride 2 puts the two cells in different residue classes, and an 8-wide stride-1 window never holds both. A swapped pair therefore stays swapped under the default preset, at any number of rounds. The quality preset's 12-wide windows happen to repair this example, but not the general problem: a mask guarantees a valid layout, not reach. The package's tests include this case.

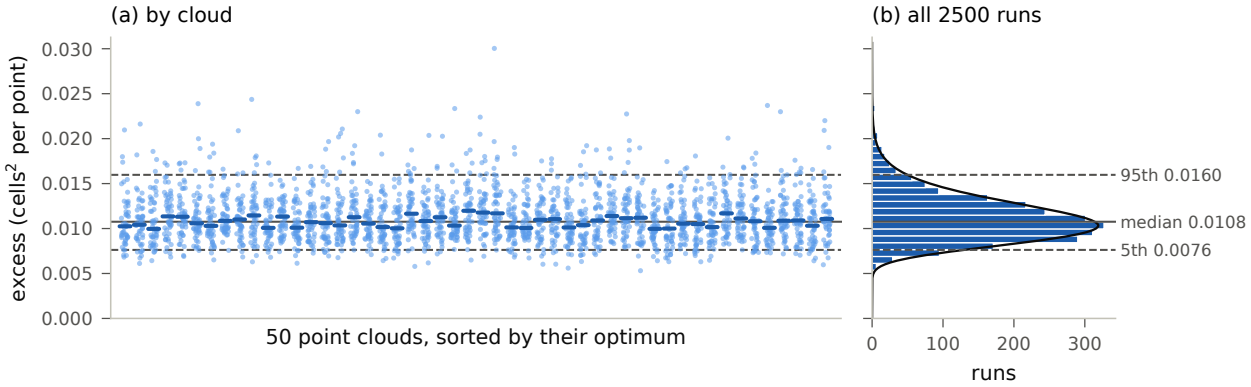


Figure 10: Excess of the default preset over the optimum, in cells^2 per point, on 50 clouds of 10^4 uniform points, each run from 50 random starts. (a) Each column is one cloud, its dots are the 50 starts and its bar is their median; clouds are sorted by their optimum. (b) All 2,500 runs, with a fitted log-normal curve. The solid line is the median, the dashed lines the 5th and 95th percentiles; every cloud spans about the same band.

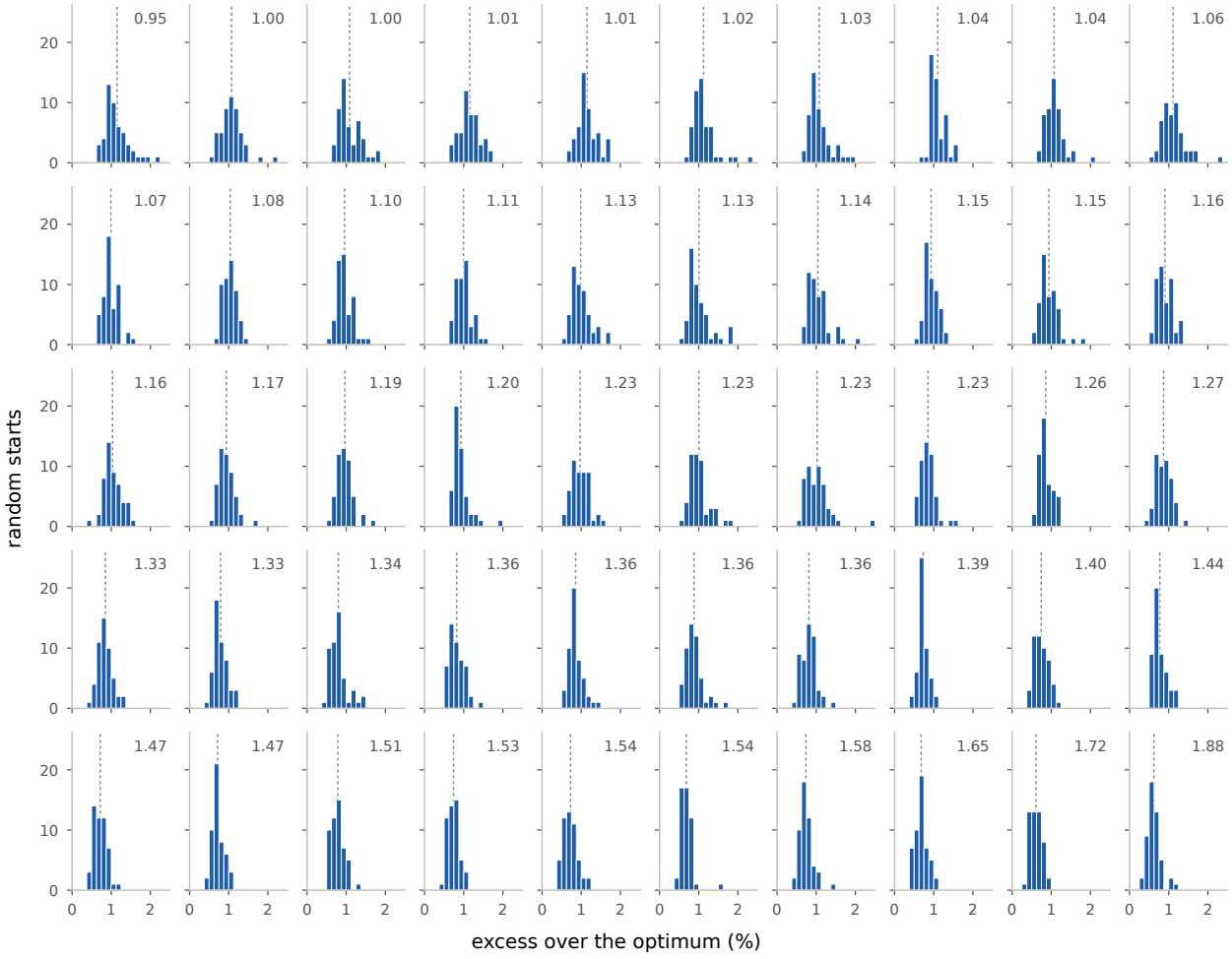


Figure 11: The runs of Figure 10, one histogram per cloud over its 50 random starts, on shared axes. Clouds are sorted by their optimum, printed in each panel in cells^2 per point; the dashed line is the mean over the starts. The histograms keep the same width and shape from cloud to cloud and move left as the optimum grows, because the absolute excess stays about the same.

7.9 Spread across inputs and random starts

The constructed fixed point shows how bad a fixed point can be, but it starts from a layout that the method never chooses. To see how much the result depends on the input and on the random start in practice, we drew 50 clouds of 10^4 uniform points (instance seeds 100–149, used nowhere else), the hardest common input, and ran the default preset from 50 random starts on each (method seeds 0–49), 2,500 runs in all. Each cloud's optimum came from dense SciPy. In addition, the certified solver, warm-started from each of the 2,500 ShuffleSnap layouts, reached the same optimum every time.

From a random start, the result is a draw from one distribution, and that distribution is nearly the same for every cloud. Measured in absolute terms, as cells^2 per point added to the optimum, 96% of the variance comes from the start and 4% from the cloud (Figure 10a); only 7 of the 50 clouds' distributions differ from the pooled one at the 5% level, where 2.5 would by chance. The distribution is right-skewed rather than normal: within a cloud the skewness is 0.93, 31 of 50 clouds fail a Shapiro–Wilk test, and 23 runs lie more than three standard deviations above their cloud's mean, where a normal distribution would put 3.4. On a log scale it is close to normal (skewness 0.28, 5 of 50 clouds fail the test). The pooled runs fit a log-normal distribution with a median of $0.0109\,\text{cells}^2$ per point and $\sigma = 0.23$ (Figure 10b), and 90% of runs fall between 0.71 and 1.48 times that median.

In relative terms, the excess had a mean of 0.91% (standard deviation 0.26%) and a median of 0.87%, and 90% of runs fell between 0.55% and 1.38%. The extremes were 0.32% and 2.44%. Even the worst run left 75% of its points on their optimal cells (82% on average). Because the absolute excess hardly depends on the cloud while the optimum varies by a factor of two (0.95–$1.88\,\text{cells}^2$ per point), clouds with a larger optimum show a smaller relative excess: the correlation between a cloud's optimum and its mean relative excess is -0.94, and -0.04 for the absolute excess. Each cloud's histogram of relative excess therefore has the same shape and moves left as its optimum grows (Figure 11). The start accounts for 67% of the variance of the relative excess; on one cloud, the 50 starts span a median of 0.99 percentage points.

For a user, one run at this size is a sample from a known, narrow distribution. Keeping the best of several starts helps less than a larger window: the best of 2, 4 and 10 starts averages 0.79%, 0.72% and 0.65%, while the quality preset reaches 0.28% at about three times the default's time (Table 8). The constructed fixed point, whose relative excess is unbounded, lies far outside this distribution. This is evidence, not a bound: it covers one size and one distribution, and on uniform points the excess grows with N (Section 7.4).

7.10 Parallel scaling

Figure 12 shows parallel scaling. ShuffleSnap doubles its speed with each doubling of threads up to 4, and reaches 5.8–7.0× at 8 threads, 8.6–11.2× at 16 and 10.1–13.7× at 24, with the same assignment at every thread count. The fast preset, with the least work per phase, scales least. BSP-OT reaches about 5–6× at 16 threads and gains nothing from the efficiency cores. We did not profile where ShuffleSnap loses efficiency beyond 8 threads. The efficiency cores are slower than the performance cores, and each phase ends with a synchronization point, whose share grows as the phases get cheaper.

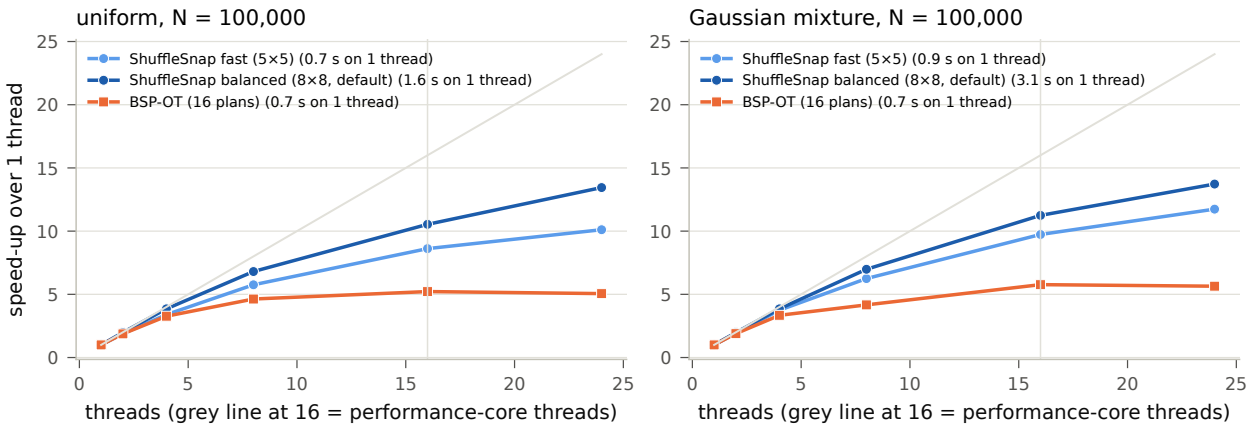


Figure 12: Speed-up over one thread at $N = 100{,}000$ (median of one run per setting, threads not pinned). The machine has 16 performance-core and 8 efficiency-core hardware threads; the light diagonal is linear speed-up. ShuffleSnap's windows are independent within a phase, so it keeps scaling with the efficiency cores. BSP-OT stops improving at 8–16 threads, because its plans run in parallel but the merge that combines them is largely sequential.

8 Discussion and limitations

Where it works. Coarse-to-fine strided windows get very close to the optimum for the smooth, clustered layouts that motivate the problem. On the two real 70,000-point UMAP embeddings, the default preset is within 0.002% of the certified optimum and visually indistinguishable from it. The method needs no dense matrix, runs in 0.15–0.4 s with all 24 threads at 10^5 points (default preset), gives the same result for any number of threads, and never increases the cost. At the running time of BSP-OT with 16 or 64 plans, it is closer to the optimum on eight or nine of ten distributions.

Where it fails. ShuffleSnap is a local search, and its failures follow from that. (i) Near-uniform data stay hardest: the default is 0.84% above the optimum at 10^5 points, and its excess grows with N (0.1% at 10^3 points, 1.3% at 10^6). The excess consists of a few long closed loops of one-cell shifts that no window can see whole. Constructed starts make the relative excess unbounded, but a random start gives a draw from a narrow distribution that hardly depends on the input: at 10^4 uniform points, 90% of 2,500 runs were 0.55–1.38% above the optimum. (ii) Masks with concave bays and grids with surplus cells need global rerouting of mass, which coarse schedules get wrong: on the S-shaped mask, points drawn inside the S end up lined along its bays. The default's larger windows and $\sqrt{2}$ schedule reduce this (1.4% on the S with uniform points, against 9.3% for BSP-OT), but surplus cells remain hard (3.1% with less than a row of surplus), and a mask can even cut exchanges between usable cells off entirely. (iii) Structured starts do not help: bisection starts hurt on every input, and row-sorted starts on most.

The exact solver changes the question. We built the certified solver as an evaluation tool. It turned out to be practical. Starting from ShuffleSnap's answer, it proved optimality at 10^5 points in 17 s to 23 min on one core (median about 2 min), and at 10^6 uniform points in 11 min. When a certified optimum is worth the wait, it is the better choice at this scale, and ShuffleSnap's role is then that of a fast, good-enough answer, a warm start, and a method for sizes and time budgets where exact solving is out of reach. We report this even though it narrows the case for the method we set out to test.

Displacement is not neighbourhood. On the real embeddings, BSP-OT comes within 2–9% of the optimal displacement, yet its layouts are locally scrambled (Figure 6, row 2), and it halves both neighbourhood measures. RasterFairy keeps local order inside its blocks and breaks it along block seams. Even the optimum tears neighbourhoods along cluster boundaries (Figure 6, row 5). If neighbourhood preservation is the goal, it should be optimized or at least measured directly, e.g. with LAS/FLAS and DPQ [2]. A lower displacement does not guarantee it.

Which method to use. In our measurements:

- If a proven optimum is worth minutes of computation, use the certified solver started from ShuffleSnap. We tested it up to 10^6 uniform and 2×10^5 clustered points.
- For interactive use, the default preset gives near-optimal layouts of 10^5 points in about 0.2 s on a multicore machine, or 2.3 s on one core; the fast preset takes 0.09 s or 0.77 s, about as long as BSP-OT with 16 plans, and is closer to the optimum on eight of ten distributions.
- Under very tight budgets (below 0.12–0.36 s on one core at 10^5 points), BSP-OT gives the best result available. With 24 threads we found no such budget.
- When the position of empty cells does not matter, an exact-fit mask is both simpler and handled far better than surplus cells, by every method.

Limitations of this study. Everything ran on one workstation, CPU only. BSP-OT has a CUDA implementation that we did not test. The objective is squared Euclidean displacement in 2D. The window solver is exact for any cost, but its geometric start is derived for squared distances, and we did not evaluate other costs or 3D grids. The certified solver depends on OR-Tools and on integer scaling. Its certificate accounts for the scaling, but its running time depends on the candidate set and the warm start. Real data consist of two UMAP embeddings with one parameter setting. The billion-point run is a single instance and has no known optimum. The bbox normalization is part of the problem definition; other normalizations change the optimum. Timings exclude warm-up (imports, thread start-up) and follow one pinning policy, and unrelated background services were running (load about 1). We compared against the representative settings of each comparator, but not every setting. In particular, RasterFairy's swap optimizer was tested only at $N = 2{,}500$: it needs dense $N \times N$ memory, and its PyPI release does not run on Python 3.

9 Conclusion

The Shellsort idea largely works. Strided exact windows, refined from coarse to fine, bring clustered layouts, including two real UMAP embeddings of 70,000 points, to within hundredths of a percent of a proven optimum in a fraction of a second on a multicore machine, with little memory and deterministic parallelism. The idea also has sharp limits. Its remaining error forms long loops that no window can see, which makes uniform data, lattices, concave masks and surplus cells hard, with an unbounded worst case. Window size matters most, and a window solver that starts from closed-form transport duals lets ShuffleSnap use windows large enough to beat BSP-OT at equal running time. The most consequential finding may be the exact solver built to judge the heuristic: seeded with ShuffleSnap's answer, proven optima are within reach at 10^5–10^6 points. For point-to-grid layout, exactness is therefore a matter of minutes, not a matter of impossibility.

References

[1] Franz Aurenhammer, Friedrich Hoffmann, and Boris Aronov. Minkowski-type theorems and least-squares clustering. *Algorithmica*, 20(1):61–76, 1998. doi: 10.1007/PL00009187. URL https://doi.org/10.1007/PL00009187.

[2] Kai Uwe Barthel, Nico Hezel, Klaus Jung, and Konstantin Schall. Improved evaluation and generation of grid layouts using distance preservation quality and linear assignment sorting. *Computer Graphics Forum*, 42(1):261–276, 2023. doi: 10.1111/cgf.14718. URL https://doi.org/10.1111/cgf.14718.

[3] Dimitri P. Bertsekas. A distributed algorithm for the assignment problem. Working paper, Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, Cambridge, MA, March 1979. URL https://web.mit.edu/dimitrib/www/Bertsekas_Auction_Distributed_1979.pdf. Original auction algorithm. Retyped copy hosted on the author's MIT page.

[4] Dimitri P. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14(1):105–123, 1988. doi: 10.1007/BF02186476. URL https://doi.org/10.1007/BF02186476.

[5] Nicolas Bonneel, Julien Rabin, Gabriel Peyré, and Hanspeter Pfister. Sliced and Radon Wasserstein barycenters of measures. *Journal of Mathematical Imaging and Vision*, 51(1):22–45, 2015. doi: 10.1007/s10851-014-0506-3. URL https://doi.org/10.1007/s10851-014-0506-3.

[6] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991. doi: 10.1002/cpa.3160440402. URL https://doi.org/10.1002/cpa.3160440402.

[7] Rainer Burkard, Mauro Dell'Amico, and Silvano Martello. *Assignment Problems*. Society for Industrial and Applied Mathematics, 2009. ISBN 978-0-89871-663-4. doi: 10.1137/1.9780898717754. URL https://doi.org/10.1137/1.9780898717754. First edition. A revised reprint appeared in 2012 (see burkard2012assignment).

[8] David F. Crouse. On implementing 2D rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, 52(4):1679–1696, 2016. doi: 10.1109/TAES.2016.140952. URL https://doi.org/10.1109/TAES.2016.140952.

[9] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In C. J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper/2013/hash/af21d0c97db2e27e13572cbf59eb343d-Abstract.html.

[10] Rémi Flamary, Cédric Vincent-Cuaz, Nicolas Courty, Alexandre Gramfort, Oleksii Kachaiev, Huy Quang Tran, Laurène David, Clément Bonet, Nathan Cassereau, Théo Gnassounou, Eloi Tanguy, Julie Delon, Antoine Collas, Sonia Mazelet, Laetitia Chapel, Tanguy Kerdoncuff, Xizheng Yu, Matthew Feickert, Paul Krzakala, Tianlin Liu, Eduardo Fernandes Montesuma, Nathan Neike, Baptiste Genest, David Coeurjolly, Thibaut Germain, Sienna O'Shea, Marco Corneli, and Ferdinand Genans. POT Python optimal transport. https://github.com/PythonOT/POT, 2026. URL https://github.com/PythonOT/POT. Version 0.9.7. Accessed 2026-09-23.

[11] Ohad Fried, Stephen DiVerdi, Maciej Halber, Elena Sizikova, and Adam Finkelstein. IsoMatch: Creating informative grid layouts. *Computer Graphics Forum*, 34(2):155–166, 2015. doi: 10.1111/cgf.12549. URL https://doi.org/10.1111/cgf.12549. Proc. Eurographics 2015.

[12] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Sparse transport plans between discrete measures in loglinear time. *ACM Transactions on Graphics*, 44(6):1–15, December 2025. doi: 10.1145/3763281. URL https://doi.org/10.1145/3763281. Proc. SIGGRAPH Asia 2025.

[13] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Official source code. https://github.com/baptiste-genest/BSP-OT, 2025. URL https://github.com/baptiste-genest/BSP-OT. C++ (MIT license). A CUDA version is at https://github.com/baptiste-genest/BSP-OT-CUDA. Accessed 2026-09-23.

[14] Samuel Gerber and Mauro Maggioni. Multiscale strategies for computing optimal transport. *Journal of Machine Learning Research*, 18(72):1–32, 2017. URL https://jmlr.org/papers/v18/16-108.html.

[15] Andrew V. Goldberg and Robert Kennedy. An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, 71(2):153–177, 1995. doi: 10.1007/BF01585996. URL https://doi.org/10.1007/BF01585996.

[16] Google OR-Tools. ortools/graph/min_cost_flow.h: Source file (OR-Tools v9.15 C++ reference). https://or-tools.github.io/docs/cpp/min__cost__flow_8h_source.html. URL https://or-tools.github.io/docs/cpp/min__cost__flow_8h_source.html. States: "An implementation of a cost-scaling push-relabel algorithm for the min-cost flow problem." The companion linear_assignment.h states a cost-scaling push-relabel algorithm for the assignment problem "from the paper of Goldberg and Kennedy (1995)". Accessed 2026-09-23.

[17] Gladys M. Hilasaca, Wilson E. Marcílio-Jr, Danilo M. Eler, Rafael M. Martins, and Fernando V. Paulovich. A grid-based method for removing overlaps of dimensionality reduction scatterplot layouts. *IEEE Transactions on Visualization and Computer Graphics*, 30(8):5733–5749, 2024. doi: 10.1109/TVCG.2023.3309941. URL https://doi.org/10.1109/TVCG.2023.3309941.

[18] R. Jonker and A. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. URL https://doi.org/10.1007/BF02278710.

[19] Tomas Kazmar et al. lap: Linear assignment problem solver (LAPJV/LAPMOD). https://github.com/gatagat/lap. URL https://github.com/gatagat/lap. Python package (BSD-2-Clause). Latest PyPI version 0.5.13 (2026-02-23). Accessed 2026-09-23.

[20] Daniel A. Keim and Annemarie Herrmann. The Gridfit algorithm: An efficient and effective approach to visualizing large amounts of spatial data. In *Proceedings Visualization '98*, pages 181–188. IEEE, 1998. doi: 10.1109/VISUAL.1998.745301. URL https://doi.org/10.1109/VISUAL.1998.745301.

[21] Mario Klingemann. RasterFairy. https://github.com/Quasimondo/RasterFairy, 2015. URL https://github.com/Quasimondo/RasterFairy. GitHub repository created 2016-01-22; latest PyPI release of `rasterfairy` is 1.0.6 (2019-04-29). Accessed 2026-09-23.

[22] Teuvo Kohonen. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69, 1982. doi: 10.1007/BF00337288. URL https://doi.org/10.1007/BF00337288.

[23] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109. URL https://doi.org/10.1002/nav.3800020109.

[24] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL https://doi.org/10.1109/5.726791.

[25] Bruno Lévy. A numerical algorithm for L_2 semi-discrete optimal transport in 3D. *ESAIM: Mathematical Modelling and Numerical Analysis*, 49(6):1693–1715, 2015. doi: 10.1051/m2an/2015055. URL https://doi.org/10.1051/m2an/2015055.

[26] Vadim Markovtsev. Jonker-Volgenant algorithm + t-SNE = super powers. source{d} blog. Archived at http://web.archive.org/web/20180611012448/https://blog.sourced.tech/post/lapjv/, March 2017. URL http://web.archive.org/web/20180611012448/https://blog.sourced.tech/post/lapjv/. Posted 14 March 2017. Accessed 2026-09-23 via the Internet Archive.

[27] Vadim Markovtsev and source{d}. lapjv: Linear assignment problem solver using Jonker-Volgenant algorithm. https://github.com/src-d/lapjv, 2017. URL https://github.com/src-d/lapjv. Python 3 module (MIT license). First PyPI release 1.0.0 (2017-01-09); latest PyPI version 1.3.29 (2026-09-22). Accessed 2026-09-23.

[28] Kyle McDonald. CloudToGrid. https://github.com/kylemcdonald/CloudToGrid, 2016. URL https://github.com/kylemcdonald/CloudToGrid. Jupyter notebook mapping a t-SNE point cloud to a 2D grid via the assignment problem (lapjv and hungarian packages). Accessed 2026-09-23.

[29] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction, 2018. URL https://arxiv.org/abs/1802.03426.

[30] Leland McInnes, John Healy, Nathaniel Saul, and Lukas Großberger. UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018. doi: 10.21105/joss.00861. URL https://doi.org/10.21105/joss.00861.

[31] Quentin Mérigot. A multiscale approach to optimal transport. *Computer Graphics Forum*, 30(5):1583–1592, 2011. doi: 10.1111/j.1467-8659.2011.02032.x. URL https://doi.org/10.1111/j.1467-8659.2011.02032.x.

[32] James Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957. doi: 10.1137/0105003. URL https://doi.org/10.1137/0105003.

[33] Laurent Perron and Vincent Furnon. OR-Tools. https://developers.google.com/optimization/, 2026. URL https://developers.google.com/optimization/. Version v9.15, released 2026-01-12. Accessed 2026-09-23.

[34] Gabriel Peyré and Marco Cuturi. Computational optimal transport with applications to data sciences. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/2200000073. URL https://doi.org/10.1561/2200000073.

[35] Novi Quadrianto, Le Song, and Alex J. Smola. Kernelized sorting. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc., 2008. URL https://proceedings.neurips.cc/paper_files/paper/2008/hash/94c7bb58efc3b337800875b5d382a072-Abstract.html.

[36] Novi Quadrianto, Alex J. Smola, Le Song, and Tinne Tuytelaars. Kernelized sorting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(10):1809–1821, 2010. doi: 10.1109/TPAMI.2009.184. URL https://doi.org/10.1109/TPAMI.2009.184.

[37] Julien Rabin, Gabriel Peyré, Julie Delon, and Marc Bernot. Wasserstein barycenter and its application to texture mixing. In Alfred M. Bruckstein, Bart M. ter Haar Romeny, Alexander M. Bronstein, and Michael M. Bronstein, editors, *Scale Space and Variational Methods in Computer Vision (SSVM 2011)*, volume 6667 of *Lecture Notes in Computer Science*, pages 435–446, Berlin, Heidelberg, 2012. Springer. doi: 10.1007/978-3-642-24785-9_37. URL https://doi.org/10.1007/978-3-642-24785-9_37.

[38] Sharath Raghvendra and Pankaj K. Agarwal. A near-linear time ε-approximation algorithm for geometric bipartite matching. *Journal of the ACM*, 67(3):1–19, 2020. doi: 10.1145/3393694. URL https://doi.org/10.1145/3393694.

[39] Bernhard Schmitzer. A sparse multiscale algorithm for dense optimal transport. *Journal of Mathematical Imaging and Vision*, 56(2):238–259, 2016. doi: 10.1007/s10851-016-0653-9. URL https://doi.org/10.1007/s10851-016-0653-9.

[40] R. Sharathkumar and Pankaj K. Agarwal. Algorithms for the transportation problem in geometric settings. In *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 306–317. Society for Industrial and Applied Mathematics, 2012. doi: 10.1137/1.9781611973099.29. URL https://doi.org/10.1137/1.9781611973099.29.

[41] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959. doi: 10.1145/368370.368387. URL https://doi.org/10.1145/368370.368387.

[42] Grant Strong and Minglun Gong. Self-sorting map: An efficient algorithm for presenting multimedia data in structured layouts. *IEEE Transactions on Multimedia*, 16(4):1045–1058, 2014. doi: 10.1109/TMM.2014.2306183. URL https://doi.org/10.1109/TMM.2014.2306183.

[43] Kenshi Takayama. bspot-python: Python binding to BSP-OT. https://github.com/kenshi84/bspot-python, 2026. URL https://github.com/kenshi84/bspot-python. PyPI package `bspot`, latest version 1.2.0 (2026-01-28), MIT license. Accessed 2026-09-23.

[44] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL https://jmlr.org/papers/v9/vandermaaten08a.html.

[45] Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In Georg Dorffner, Horst Bischof, and Kurt Hornik, editors, *Artificial Neural Networks — ICANN 2001*, volume 2130 of *Lecture Notes in Computer Science*, pages

485–491, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44668-0_68. URL https://doi.org/10.1007/3-540-44668-0_68.

[46] Jarkko Venna and Samuel Kaski. Local multidimensional scaling. *Neural Networks*, 19(6–7):889–899, 2006. doi: 10.1016/j.neunet.2006.05.014. URL https://doi.org/10.1016/j.neunet.2006.05.014.

[47] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2. URL https://doi.org/10.1038/s41592-019-0686-2.

[48] A. Volgenant. Linear and semi-assignment problems: A core oriented approach. *Computers & Operations Research*, 23(10):917–932, 1996. doi: 10.1016/0305-0548(96)00010-X. URL https://doi.org/10.1016/0305-0548(96)00010-X.

[49] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. URL https://arxiv.org/abs/1708.07747.

A Reproducing the results

The paper repository (`github.com/kylemcdonald/shufflesnap-paper`) contains every script, raw result, and figure source. The implementation is a separate repository (`github.com/kylemcdonald/shufflesnap`); the benchmarked code is identical, apart from its version string, to release 0.4.0 on PyPI (`pip install shufflesnap`). Starting from a checkout of both side by side:

```
pip install ./shufflesnap[exact]   # C++ build, ~2 s
pip install -r shufflesnap-paper/requirements.txt
cd shufflesnap-paper
python experiments/make_embeddings.py mnist fashion   # UMAP, fixed seed
python experiments/references_all.py 6                # certified optima (parallel)
bash experiments/make_frozen_snapshot.sh              # v0.4.0 snapshot under test
bash experiments/run_all.sh                           # all benchmarks, sequential
bash experiments/run_swap.sh                          # RasterFairy's swap optimizer
python experiments/small_exactness.py                 # small-N exactness table
python experiments/dev_ablation.py                    # development-set search
python experiments/seed_study.py                      # spread over inputs and starts
python experiments/seed_study_report.py
python experiments/make_figures.py all && python experiments/make_tables.py
python experiments/fig_method.py; python experiments/fig_failures.py
python experiments/fig_reference.py                   # annotated reference
python experiments/fig_visual.py mnist; python experiments/fig_visual.py fashion
python experiments/fig_masks.py
cd paper && tectonic main.tex
```

Raw results are JSON lines in `results/raw/`: one record per run, holding the full run specification, timings, memory, validity check, cost, displacement statistics, and the software environment

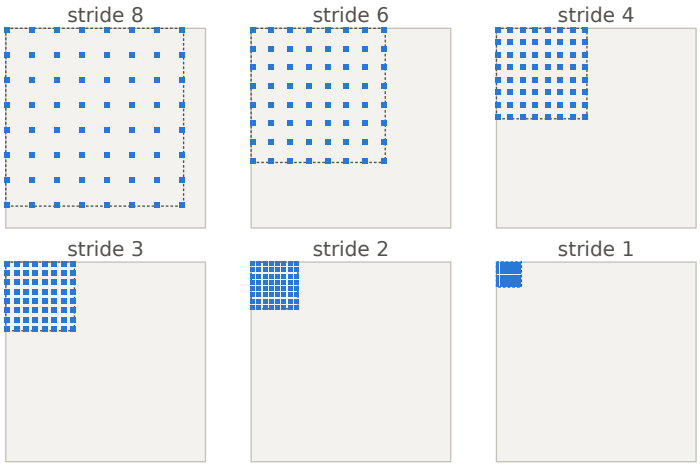
(library versions, commit hashes of the comparator repositories). Certified optima are cached in `results/reference/` with their lower bound and gap. The lab notebook `notebook/LOG.md` records decisions and negative results in the order they happened.

Timing boundary (summary). A timed run is one call of a method adapter in a fresh process, after a warm-up call on a 256-point instance. It includes method-specific preprocessing and excludes instance generation, the shared normalization, and validation. Peak memory is the rise of peak RSS during the call.

B A serial reference implementation

This is the complete algorithm: the file `shufflesnap/reference.py` from the package, split into its parts, each beside a diagram computed with its code. Without exact ties it returns the same assignment as the compiled kernel; the package's tests check this for every preset on six kinds of target, from an exact-fit square to surplus cells and a random mask. The kernel adds threads, skips windows whose cells have not changed (which does not change the result) and an optional time budget. Points are in grid units: cell (x, y) has its center at (x, y).

The schedule.



A window always holds $w \times w$ cells, but at stride s they lie s cells apart, so it spans $(w-1)s+1$ cells per axis (dashed). The first stride is the smallest power of two with $ws \geq L$, so the first windows reach across the axis; each later stride is the last one divided by `ratio` and rounded. Each axis has its own strides, and the shorter axis waits at 1. Drawn: one window of the default preset ($w = 8$) on a 64×64 grid at every stride.

```python
"""ShuffleSnap in NumPy and SciPy, written to be read: serial, unit cells.

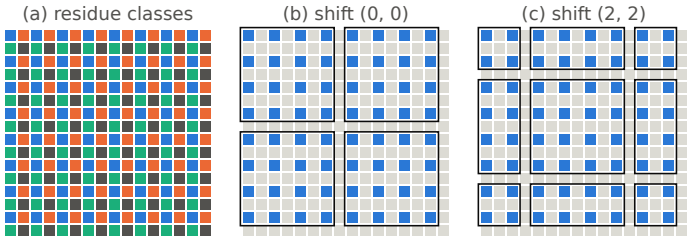
Without exact ties it returns the same cells as the compiled kernel,
``shufflesnap.assign(points, grid, normalize="none", ...)`` (tested).
``snap(P, mask)`` is the default preset; ``w=5`` and ``w=12`` give the
fast and quality presets.
"""
from collections import defaultdict
from itertools import zip_longest

import numpy as np
from scipy.optimize import linear_sum_assignment


def strides(length, w, ratio):
    """Strides on one axis: the first window spans it, then divide by ratio."""
    s = 1
    while w * s < length:
        s *= 2
    out = []
    while s > 1 + 1e-9:
        out.append(max(1, round(s)))
        s /= ratio
    return out


def schedule(width, height, w, ratio):
    """Stride pairs (sx, sy) before stride 1; the shorter axis waits at 1."""
    sx, sy = strides(width, w, ratio), strides(height, w, ratio)
    pairs = []
    for pair in zip_longest(sx, sy, fillvalue=1):
        if pair != (1, 1) and pair not in pairs[-1:]:
            pairs.append(pair)
    return pairs
```

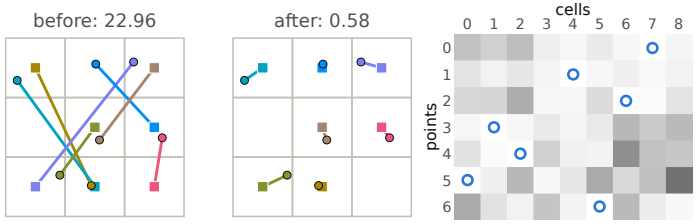
The windows of one tiling.



At stride (s_x, s_y) the cells fall into $s_x s_y$ residue classes ($x \bmod s_x$, $y \bmod s_y$) (a). In its class, cell x is lattice column $x // s_x$, and tiles of w lattice columns start at column `ox`. A window is the set of cells that share a class and a tile (b, boxes). The four tilings of a round shift the tiles by half a window (c). Edge windows are smaller, and masked cells never appear. Drawn with $w = 4$, stride 2, class $(0, 0)$.

```python
def windows(cx, cy, sx, sy, w, ox, oy):
    """One tiling: cells in the same residue class and the same tile."""
    tx = (cx // sx + w - ox) // w  # tiles start at lattice column ox
    ty = (cy // sy + w - oy) // w
    groups = defaultdict(list)
    for cell, key in enumerate(zip(cx % sx, cy % sy, tx, ty)):
        groups[key].append(cell)
    return [np.array(cells) for cells in groups.values()]
```

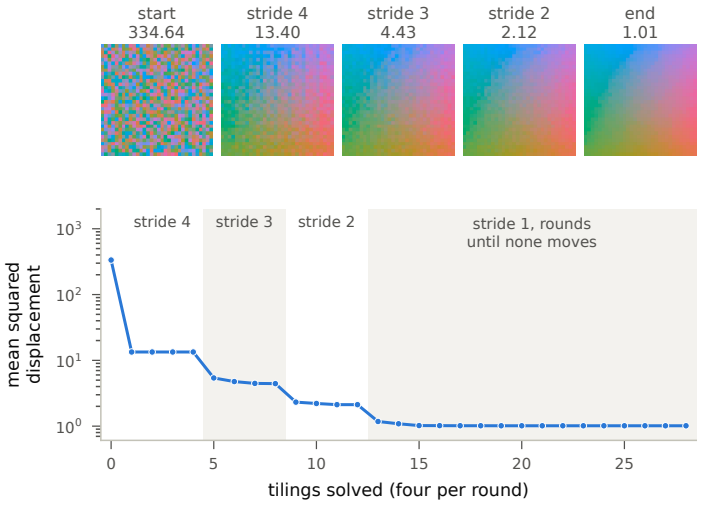
One window.



The window's points are reassigned among all of its cells, empty ones included, by an exact linear assignment on squared distances. The result is kept only if it lowers the window's cost by more than a relative `tol`, so the cost never rises and ties cannot cycle. `pos` gives each point's cell and `occ` each cell's point. Drawn: seven points in a 3×3 window before and after, with summed squared distances, and the cost matrix with the chosen entries.

```python
def solve_window(P, C, cells, occ, pos, tol):
    """Reassign the window's points among its cells; keep it if better."""
    pts = occ[cells][occ[cells] >= 0]  # the points in these cells
    cost = ((P[pts, None] - C[None, cells]) ** 2).sum(-1)  # points x cells
    new = cells[linear_sum_assignment(cost)[1]]
    old_cost = ((P[pts] - C[pos[pts]]) ** 2).sum()
    new_cost = ((P[pts] - C[new]) ** 2).sum()
    if new_cost >= old_cost * (1 - tol):  # strict improvements only
        return False
    occ[cells] = -1
    occ[new] = pts
    pos[pts] = new
    return True
```

The whole method.



The cells are the mask's usable entries in row-major order, and point i starts in a random cell `pos[i]`. A round solves the windows of the four tilings in turn. Each coarse stage runs one round; then stride-1 rounds repeat until one moves nothing, a fixed point of every window but not a proof of optimality. Drawn: 1,024 uniform points on a 32×32 grid, colored by input position, after the start, each coarse stage and the end (top), and the mean squared displacement in cells^2 after every tiling (bottom): each stride halves it or better, and the last stride-1 round moves nothing.

```python
def snap(P, mask, w=8, ratio=2 ** 0.5, seed=0, tol=1e-12):
    """Cell of each point; P is (N, 2) in grid units, cells are row-major."""
    cy, cx = np.nonzero(np.asarray(mask, dtype=bool))
    C = np.stack([cx, cy], axis=1).astype(float)  # cell centers (x, y)
    rng = np.random.default_rng(seed)
    pos = rng.permutation(len(C))[:len(P)]  # random start: cell of point
    occ = np.full(len(C), -1)  # point in each cell, -1 if empty
    occ[pos] = np.arange(len(P))
    h = w // 2

    def one_round(sx, sy):  # four tilings, shifted by half a window
        moved = 0
        for ox, oy in [(0, 0), (0, h), (h, 0), (h, h)]:
            for cells in windows(cx, cy, sx, sy, w, ox, oy):
                moved += solve_window(P, C, cells, occ, pos, tol)
        return moved

    for sx, sy in schedule(mask.shape[1], mask.shape[0], w, ratio):
        one_round(sx, sy)
    while one_round(1, 1):  # stride 1 until nothing moves
        pass
    return pos
```

C Additional tables

Table 7: Absolute excess over the certified optimum, in cells^2 per point, at $N = 100{,}000$ (same runs as Table 2). "Optimum" is the certified optimal mean squared displacement. Relative and absolute excess tell different stories on the heavy-tailed input, where outliers dominate the optimum.

distribution	optimum	SS fast	BSP 16	SS bal.	BSP 64	SS qual.	RF master	RF PyPI	Bisect.
uniform	1.80	0.045	0.80	0.015	0.58	0.008	0.80	1.10	1.81
Gaussian	6501	0.030	3.61	0.011	2.60	0.005	0.38	0.66	1.17
Gaussian mixture	4440	0.58	252	0.11	167	0.010	751	752	554
90/10 clusters	8743	4.34	2.77	1.01	1.66	0.079	102	101	49.9
ring	1731	0.081	20.1	0.022	16.5	0.008	710	710	697
spiral	1338	0.29	133	0.055	108	0.015	366	367	460
checkerboard	32.1	0.041	16.0	0.018	11.6	0.006	137	137	145
heavy tails	32818	0.009	0.019	0.007	0.012	0.005	0.018	0.070	0.059
duplicates	4136	5.88	491	1.45	228	0.47	2034	2035	1645
near-line	9876	0.98	0.81	0.21	0.20	0.033	1912	1912	1884

Table 8: As Table 2, at $N = 10{,}000$. Here the dense SciPy solver is feasible; the last column gives its time on instance seed 1 (it returns the same optimum as the certified solver).

distribution	SS fast	BSP 16	SS bal.	BSP 64	SS qual.	RF master	RF PyPI	Bisect.	exact (s)	dense (s)
uniform	2.34	27.6	0.69	17.9	0.28	31.8	82.4	86.2	0.53	10.8
Gaussian	0.003	0.13	0.001	0.093	<0.001	0.040	0.14	0.15	1.31	205
Gaussian mixture	0.014	3.78	0.004	2.54	<0.001	38.7	39.7	13.1	2.05	176
90/10 clusters	0.037	0.13	0.011	0.049	0.001	1.32	1.47	0.66	2.91	227
ring	0.018	3.78	0.007	1.56	0.005	40.5	41.6	39.4	1.92	104
spiral	0.036	10.3	0.013	7.30	0.005	24.6	29.9	31.9	2.29	84.9
checkerboard	0.72	55.4	0.28	44.8	0.090	271.4	260.7	265.2	0.53	15.9
heavy tails	<0.001	0.005	<0.001	0.003	<0.001	<0.001	0.002	0.003	3.21	352
duplicates	0.15	12.0	0.045	4.97	0.014	57.6	58.0	36.6	0.97	126
near-line	0.012	0.027	0.003	0.007	<0.001	19.4	19.8	19.5	6.92	246
median time (s)	0.057	0.063	0.15	0.25	0.42	0.22	0.029	0.003	1.99	151

Table 9: As Table 2, at $N = 50{,}000$.

distribution	SS fast	BSP 16	SS bal.	BSP 64	SS qual.	RF master	RF PyPI	Bisect.	exact (s)
uniform	2.82	45.5	1.20	36.8	0.48	40.4	65.4	114.2	5.62
Gaussian	<0.001	0.056	<0.001	0.039	<0.001	0.009	0.018	0.031	28.1
Gaussian mixture	0.009	5.47	0.001	3.83	<0.001	38.2	38.2	14.6	37.9
90/10 clusters	0.043	0.049	0.005	0.025	<0.001	1.21	1.22	0.86	59.4
ring	0.007	2.15	0.002	0.59	<0.001	41.1	41.1	42.2	35.0
spiral	0.028	11.4	0.006	7.74	0.002	27.3	27.4	25.0	40.8
checkerboard	0.24	81.6	0.085	69.4	0.035	398.0	400.1	423.4	13.6
heavy tails	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	88.7
duplicates	0.14	13.8	0.038	4.91	0.013	58.6	58.6	39.7	157
near-line	0.003	0.028	<0.001	0.003	<0.001	19.7	19.7	19.5	75.6
median time (s)	0.36	0.34	1.03	1.35	2.95	1.46	12.4	0.016	39.3

Table 10: RasterFairy's swap optimizer (upstream `rfoptimizer.py`) at $N = 2{,}500$ (50×50 grid), mean excess over the certified optimum (%) over instance seeds 1–3, one thread. RF is RasterFairy upstream; the next four columns run the swap optimizer from its layout for 10^4 to 10^7 iterations, and the last one from a random permutation, as in upstream's demo. [a]: timed on another physical core (seed 1) or on five others in parallel (seeds 2–3). The dense SciPy solver returns the optimum in a median of 2.7 s.

							RF + swap optimizer				
distribution	SS fast	SS bal.	SS qual.	BSP 16	BSP 64	RF	10^4	10^5	10^6	$10^{7\,a}$	rand. 10^6
uniform	1.56	1.05	0.30	26.8	19.3	16.6	16.6	16.2	15.2	13.9	6320
Gaussian	0.012	0.006	0.001	0.22	0.15	0.11	0.11	0.11	0.11	0.100	29.5
Gaussian mixture	0.033	0.012	0.002	6.06	3.58	38.3	33.7	28.7	18.1	10.4	39.3
90/10 clusters	0.048	0.013	0.005	0.29	0.16	1.20	1.16	1.06	0.82	0.50	17.1
ring	0.053	0.018	0.004	2.95	2.24	35.9	35.3	33.2	21.5	12.3	101.7
spiral	0.13	0.036	0.012	9.64	5.53	10.4	10.3	9.97	8.82	7.16	140.0
checkerboard	1.15	0.43	0.33	51.2	36.1	56.3	56.0	52.5	45.1	33.6	4346
heavy tails	<0.001	<0.001	<0.001	0.006	0.004	0.003	0.003	0.003	0.002	0.002	0.43
duplicates	0.17	0.050	0.015	9.03	6.77	55.6	54.0	48.7	36.7	30.2	50.1
near-line	0.015	0.004	0.002	0.070	0.020	19.2	14.3	11.6	1.65	0.81	6.98
median time (s)	0.012	0.029	0.071	0.015	0.059	0.051	0.75	6.44	63.3	613	61.7

Table 11: All held-out ablations (24 threads): mean excess over the certified optimum (%) and median time (s) over instance seeds 1–3. Each row changes one setting of the default (8×8 windows, stride ratio $\sqrt{2}$, four tilings, random start, one round per level, geometric window solver, unchanged windows skipped, automatic start stride).

change	setting	uniform 100k		mixture 100k		MNIST 70k	
		excess %	time (s)	excess %	time (s)	excess %	time (s)
window side (ratio $\sqrt{2}$)	4×4	3.76	0.057	0.027	0.062	0.068	0.044
	6×6	1.50	0.080	0.008	0.12	0.010	0.080
	8×8 (default)	0.84	0.12	0.002	0.24	0.002	0.16
	10×10	0.54	0.18	<0.001	0.48	0.002	0.30
	12×12	0.44	0.27	<0.001	0.82	<0.001	0.54
	14×14	0.27	0.39	<0.001	1.41	<0.001	0.90
	16×16	0.19	0.54	<0.001	2.19	<0.001	1.53
	20×20	0.11	1.01	<0.001	4.90	<0.001	3.08
	24×24	0.090	1.60	<0.001	9.31	<0.001	6.44
window side (halving)	4×4	6.36	0.044	0.035	0.050	0.079	0.033
	6×6	2.51	0.060	0.011	0.085	0.015	0.057
	8×8	1.52	0.097	0.004	0.17	0.004	0.12
	12×12	0.54	0.23	<0.001	0.58	0.001	0.39
	16×16	0.29	0.48	<0.001	1.49	<0.001	1.06
	24×24	0.12	1.54	<0.001	7.14	<0.001	5.81
stride ratio (8×8)	3	2.35	0.077	0.011	0.15	0.011	0.10
	2 (halving)	1.52	0.096	0.004	0.17	0.004	0.12
	1.68	0.96	0.10	0.002	0.20	0.003	0.13
	$\sqrt{2}$ (default)	0.84	0.12	0.002	0.25	0.002	0.16
	1.19	0.92	0.16	0.002	0.39	0.004	0.24
	1.1	1.21	0.20	0.002	0.49	0.006	0.30
start	random (default)	0.84	0.12	0.002	0.24	0.002	0.17
	bisection	22.9	0.16	0.014	0.28	0.009	0.18
	row sort	12.4	0.13	0.001	0.28	0.003	0.18
rounds per level	1 (default)	0.84	0.12	0.002	0.25	0.002	0.17
	2	0.70	0.17	0.002	0.39	0.002	0.25
	3	0.77	0.19	0.002	0.40	0.002	0.27
tilings per round	1	104.1	0.040	0.030	0.069	0.052	0.047
	2 (diagonal)	1.95	0.071	0.004	0.13	0.004	0.089
	4 (default)	0.84	0.12	0.002	0.24	0.002	0.17
	9	0.70	0.25	0.001	0.52	0.002	0.34
first stride	automatic (default)	0.84	0.12	0.002	0.25	0.002	0.16
	capped at 16	3.00	0.11	0.11	6.15	0.083	0.47
	capped at 8	9.62	0.21	0.12	10.0	0.16	3.73
	capped at 4	9.48	0.36	0.12	10.1	0.18	4.47
	capped at 2	12.3	0.43	0.12	11.0	0.18	4.42
	capped at 1	21.2	0.47	0.11	11.1	0.17	4.22
window solver	Hungarian, 8×8, ratio $\sqrt{2}$	0.84	0.28	0.002	1.24	0.002	0.82
	Hungarian + greedy start, 8×8, ratio $\sqrt{2}$	0.84	0.24	0.002	1.27	0.002	0.82
	Jonker–Volgenant, 8×8, ratio $\sqrt{2}$	0.84	0.75	0.002	9.07	0.002	5.43
	auction + exact finish, 8×8, ratio $\sqrt{2}$	0.84	0.32	0.002	0.61	0.002	0.39
	geometric (default), 8×8, ratio $\sqrt{2}$	0.84	0.12	0.002	0.25	0.002	0.16
	Hungarian, 12×12, ratio $\sqrt{2}$	0.44	0.74	<0.001	4.36	<0.001	2.83
	Hungarian + greedy start, 12×12, ratio $\sqrt{2}$	0.44	0.64	<0.001	4.47	<0.001	2.91
	Jonker–Volgenant, 12×12, ratio $\sqrt{2}$	0.44	4.75	<0.001	55.1	<0.001	34.0
	auction + exact finish, 12×12, ratio $\sqrt{2}$	0.44	0.66	<0.001	1.31	<0.001	0.86
	geometric (default), 12×12, ratio $\sqrt{2}$	0.44	0.26	<0.001	0.82	<0.001	0.54
skip unchanged windows	yes (default)	0.84	0.12	0.002	0.25	0.002	0.16
	no	0.84	0.16	0.002	0.32	0.002	0.21

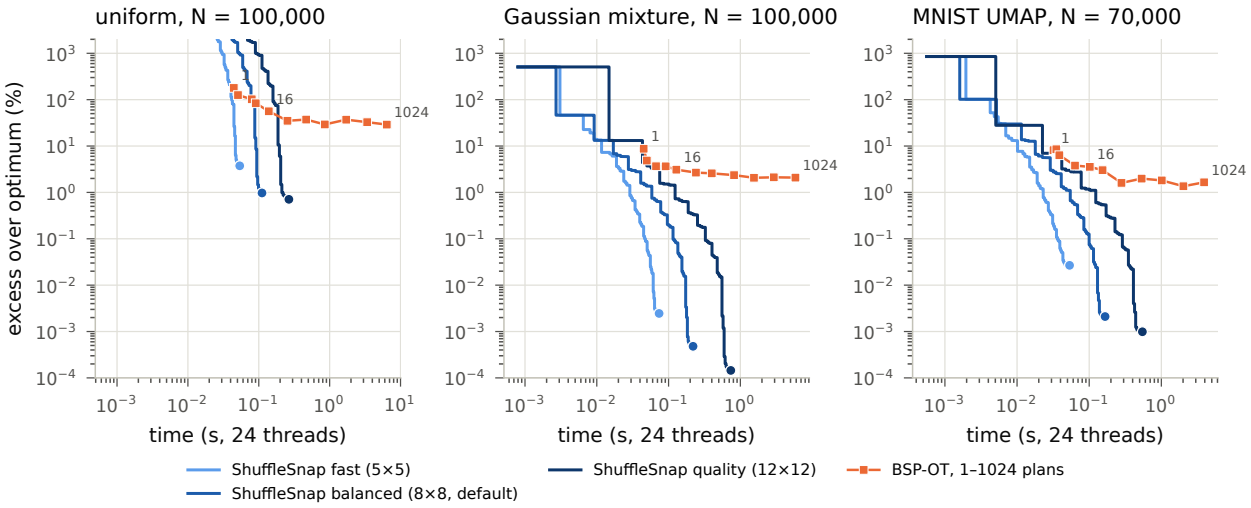


Figure 13: As Figure 3, with 24 threads for ShuffleSnap and BSP-OT (RasterFairy is single-threaded).